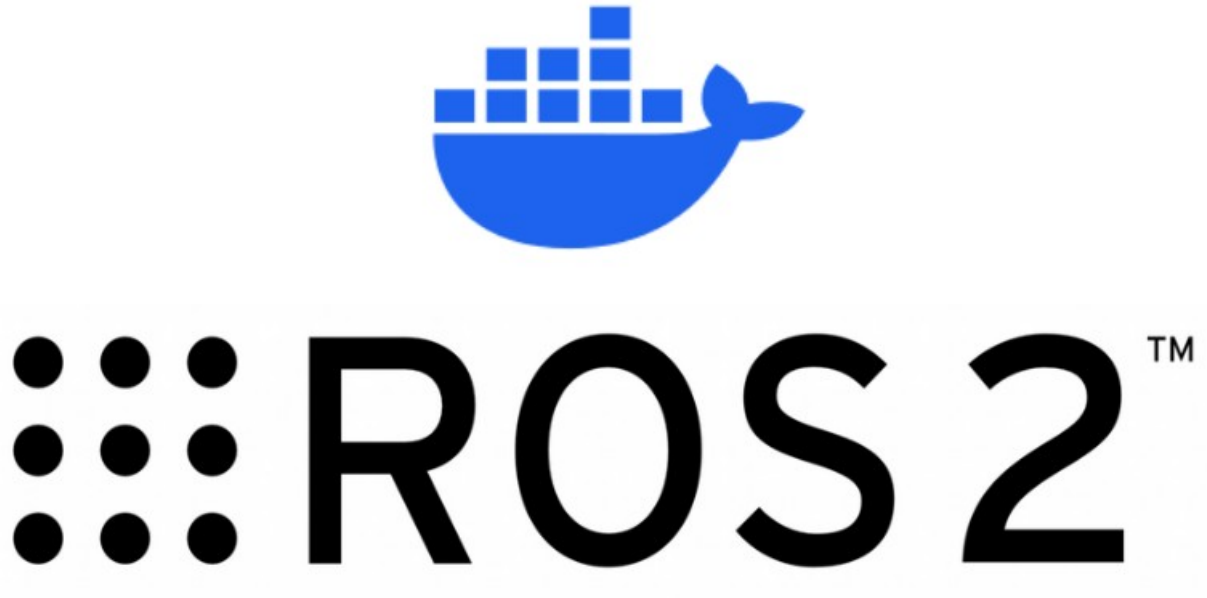


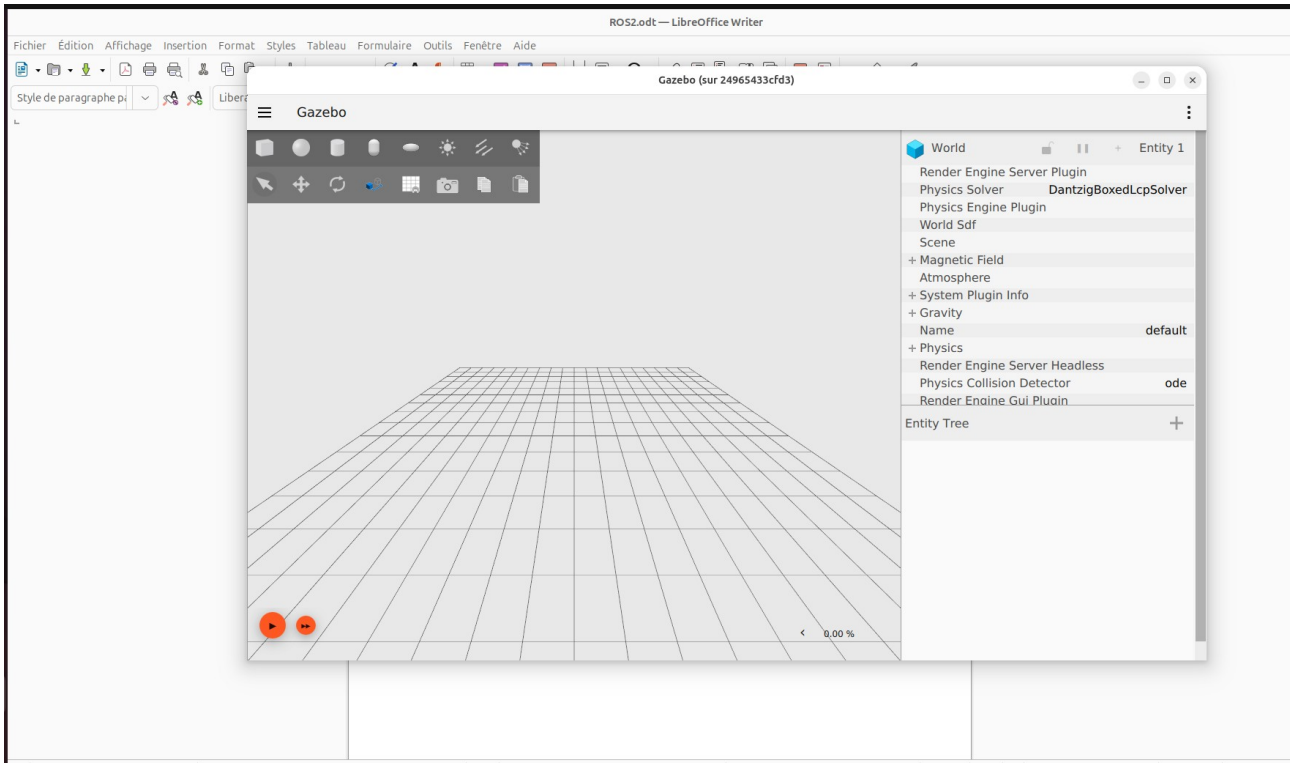
Introduction

In this part of the work, we will work on the thread 1 which is the design of a system allowing the autonomous navigation of the robot in the picking path, and to achieve this goal we will use ROS2.

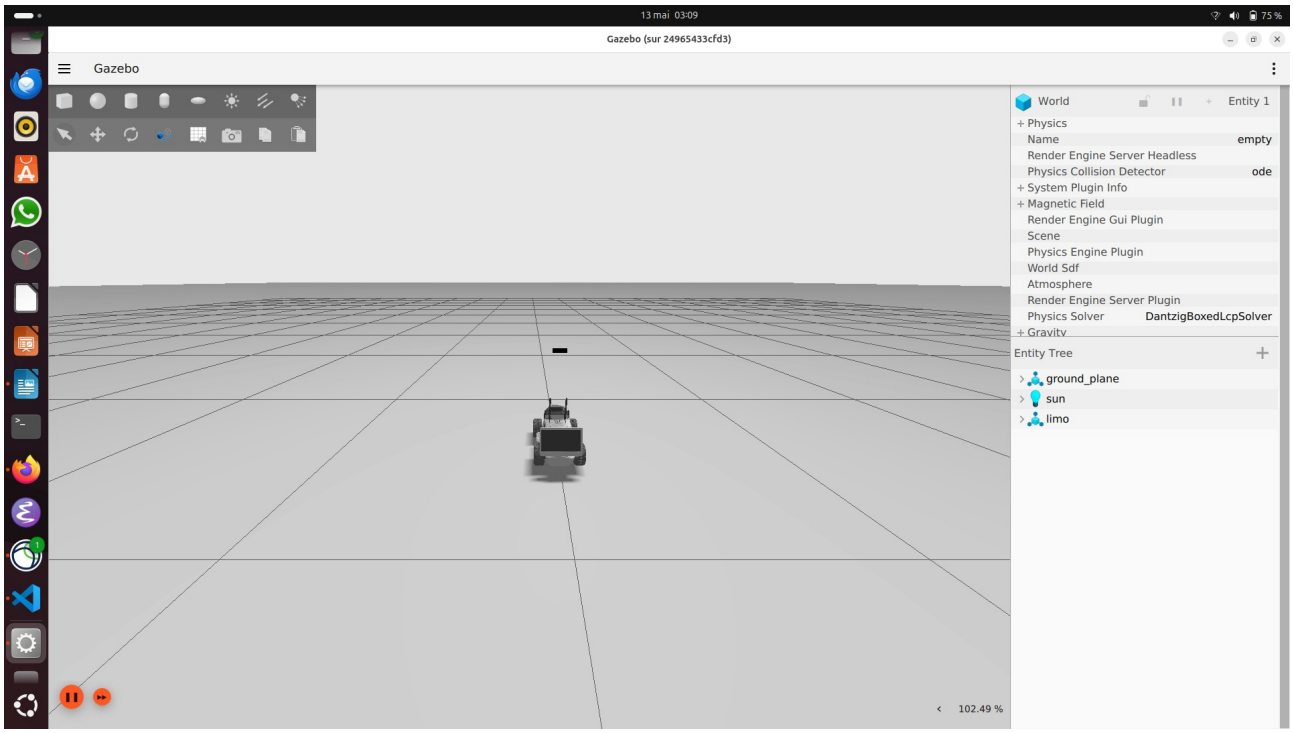
We planed to use Gazebo for the simulation of the robot but my computer doesn't run Gazebo correctly, Gazebo is slow and the optimisation to make it rapid makes the car disappear. But Gazebo is not mandatory for the project, it is true that it can be useful, but we can use the ressources ROS2 gives for the robot simulation.

II-Gazebo installation :

After the configuration and the installation of the ressources related to Gazebo, I got the following window of Gazebo.



Result of the first simulation :



There are two problem with the simulation, the first one is that the car is Limo 01 instead of 02, and the second one is that the simulation is very slow on my computer, the alternative was to use the Lab computer but I found out that the Lab has only screens not the central processing unit. Finally, I plan to use the ressources given by ROS2 and give up Gazebo, like Rviz.

III-Get starting with ROS2

A- How to run ROS2

Here are the steps to runs ROS2 and use it :

On host terminal (Ubuntu 25.04) :

1- Allow the screen acces for Gzebo

xhost +local:docker

2- Checking if containers exist :

docker ps -a

3- Start the container :

docker start limo_navigation

4- verify if the container is running :

docker ps

In the container :

1- enter the container :

docker exec -it limo_navigation bash

2- sourcing the container :

source /opt/ros/humble/setup.bash

(to source foxy we replace humble by fowy)

Listener :

- To listen :

ros2 topic echo /chatter std_msgs/msg/String

Subscriber :

-To send the data :

ros2 topic pub --rate 1 /chatter std_msgs/msg/String "{data: 'LIMO robot is ready'}"

B- Running executables

- To see the packages: ros2 pkg list

```
root@593aa2fb8316:/# ros2 pkg list
action_msgs
action_tutorials_cpp
action_tutorials_interfaces
action_tutorials_py
actionlib_msgs
ament_cmake
ament_cmake_auto
ament_cmake_copyright
ament_cmake_core
ament_cmake_cppcheck
ament_cmake_cpplint
ament_cmake_export_definitions
ament_cmake_export_dependencies
ament_cmake_export_include_directories
ament_cmake_export_interfaces
ament_cmake_export_libraries
ament_cmake_export_link_flags
ament_cmake_export_targets
ament_cmake_flake8
ament_cmake_gen_version_h
ament_cmake_gmock
```

- To see the executables : ros2 pkg executables

```
root@593aa2fb8316:/# ros2 pkg executables
action_tutorials_cpp fibonacci_action_client
action_tutorials_cpp fibonacci_action_server
action_tutorials_py fibonacci_action_client
action_tutorials_py fibonacci_action_server
composition dlopen_composition
composition linktime_composition
composition manual_composition
demo_nodes_cpp add_two_ints_client
demo_nodes_cpp add_two_ints_client_async
demo_nodes_cpp add_two_ints_server
demo_nodes_cpp allocator_tutorial
demo_nodes_cpp content_filtering_publisher
demo_nodes_cpp content_filtering_subscriber
demo_nodes_cpp even_parameters_node
```

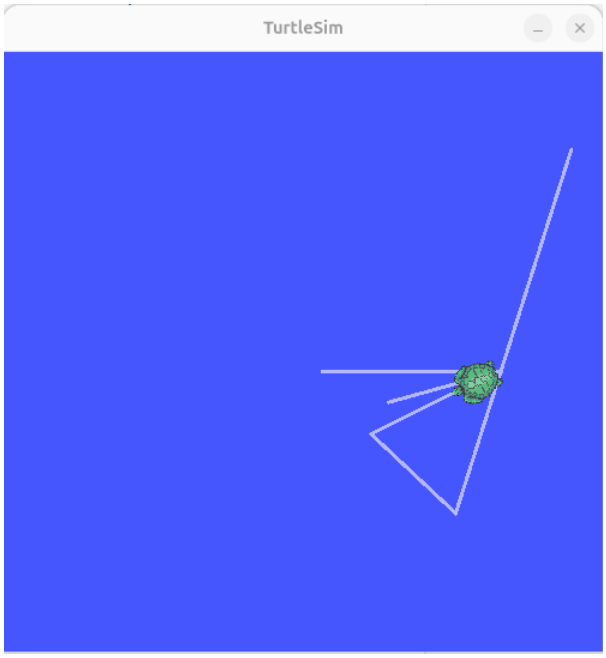
- To see the executable of the package Turtle : ros2 pkg executables turtlesim

```
root@593aa2fb8316:/# ros2 pkg executables turtle
sim
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
root@593aa2fb8316:/#
```

-To run an executable : ros2 run package_name executable_name

```
root@593aa2fb8316:/# ros2 run turtlesim turtlesim_node
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runti
me-root'
[INFO] [1779068178.065770992] [turtlesim]: Starting turtlesim with
 node name /turtlesim
[INFO] [1779068178.072032420] [turtlesim]: Spawning turtle [turtle
1] at x=[5.544445], y=[5.544445], theta=[0.000000]
amdgpu: os_same_file_description couldn't determine if two DRM fds
 reference the same file description.
If they do, bad things may happen!
amdgpu: drmGetDevice2 failed.
```

We can execute many executables at the same time :
for example, here I execute the executables turtlesim_node to show the windows and turtle_teleop_key to move the turtle from my keyboard.



ROS2 works by making nodes communicate between them, and to do it, it use topics, actions or services.
We can know what node are communicating with each other when a communication is established.
We use : **ros2 node list**

```
root@593aa2fb8316:/# source /root/ros2_ws/install/setup.bash
root@593aa2fb8316:/# ros2 node list
/teleop_turtle
/turtlesim
```

TOPICS

In the topics we have a Publisher (sender) and a Subscriber (receiver), they communicate between topics.

SERVICES

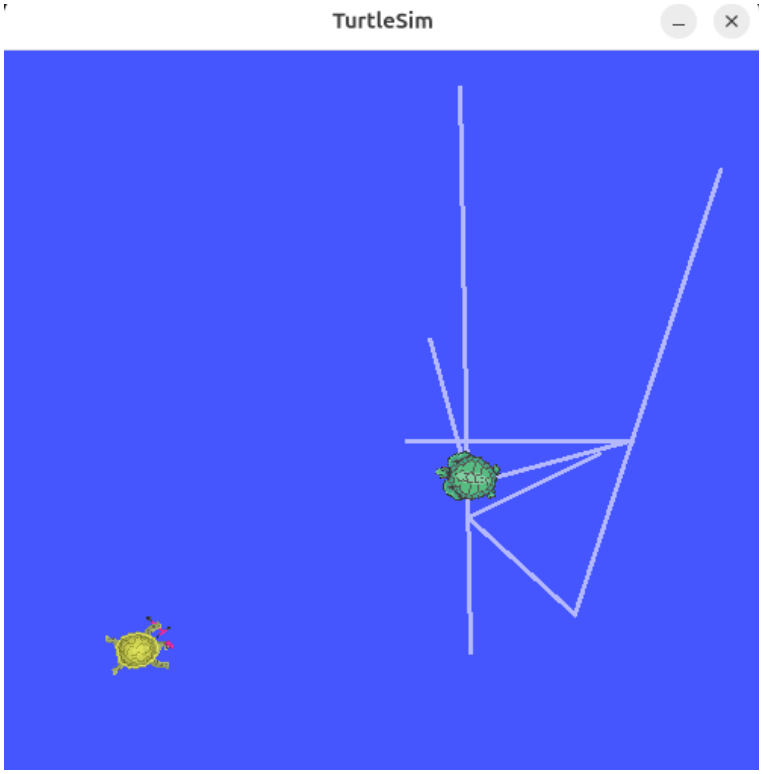
In the services we have a Server and a Client where the client require some data and the server need to respond before the client receive the data. It's mean that the server is the one which will decide if the client will receive or not the data.

Calling a services : In this case, we call for the spawning of a new turtle.

```
root@593aa2fb8316:/# ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2 , theta: 0.2, name: ''}"
2026-05-18 02:49:20.579 [RTPS_TRANSPORT_SHM Error] Failed init_port fastrtps_port7415: open_and_lock_file fai
led -> Function open_port_internal
requester: making request: turtlesim.srv.Spawn_Request(x=2.0, y=2.0, theta=0.2, name='')

response:
turtlesim.srv.Spawn_Response(name='turtle2')
```

We can see the response in the terminal.

Result :



PARAMETERS

There are used to input data into the nodes. We pass parameters in the nodes.
We can see the differents parameters by taping : **ros2 param list.**

```
root@593aa2fb8316:/# ros2 param list
2026-05-18 03:02:47.266 [RTPS_TRANSPORT_SHM Error] Failed init_port fastrtps_port7415: open_and_lock_file fai
led -> Function open_port_internal
/teleop_turtle:
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  scale_angular
  scale_linear
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  use_sim_time
```

As we can see, there are two different types of parameters: those of **/teleop_turtle** and those of **/turtlesim.** This is because there are two running executables.

-To get a parameter value : ros2 param get node_name parameter_name
for example : ros2 param get /turtlesim background_g

```
root@593aa2fb8316:/# ros2 param get /turtlesim background_g
2026-05-18 03:10:55.221 [RTPS_TRANSPORT_SHM Error] Failed init_port fastrtps_port7415: open_and_lock_file fai
led -> Function open_port_internal
Integer value is: 86
```

What is the error in red ?

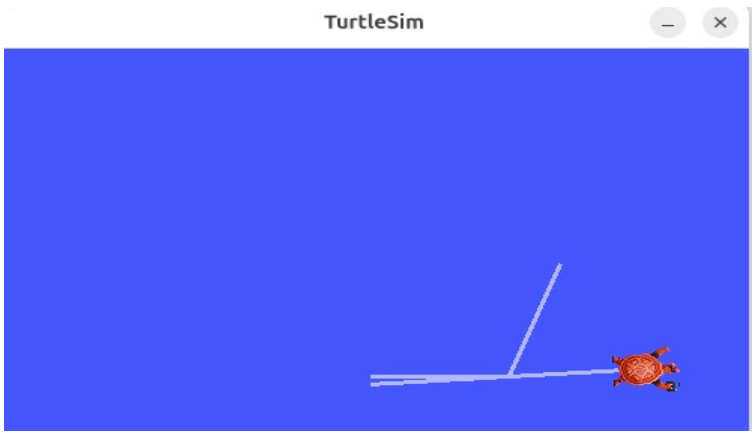
This is due that I am in docker. When we send a request, the FastDDS looks for the fastest possible way to talk to other node. And first is tries to use the Shared Memory (SHM) which allow to talk to other node using the RAM. As we are into Docker, Docker isolate the container so FastDDS can not acces the SHM and then display this error in red.

It’s just an error for trying to access to a non-autorized memory space.

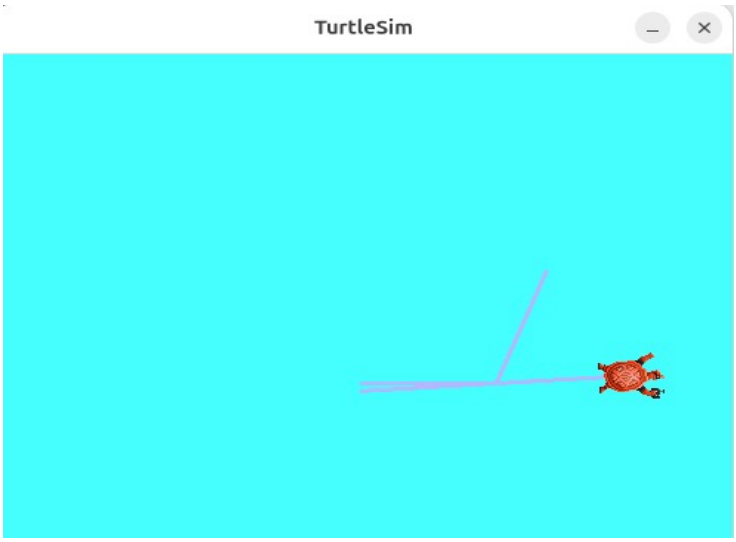
To get a parameter value :

```
root@593aa2fb8316:/# ros2 param get  /turtlesim background_g
Integer value is: 86
root@593aa2fb8316:/# ros2 param get  /turtlesim background_r
Integer value is: 69
root@593aa2fb8316:/# ros2 param get  /turtlesim background_b
Integer value is: 255
root@593aa2fb8316:/#
```

the current background is blue because the value of the parameter background_g (which is 255) is the biggest.



-To set a parameter value : ros2 param set /turtlesim background_g 255



The background has changed because we parameter background_g has changed.

- To see all the parameters for a node : ros2 param dump /turtlesim

```
root@593aa2fb8316:/# ros2 param dump /turtlesim
/turtlesim:
  ros__parameters:
    background_b: 255
    background_g: 255
    background_r: 69
    qos_overrides:
      /parameter_events:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last
          reliability: reliable
    use_sim_time: false
```

The parameter state is also given. We can put the parameters displaying in a file of the type .yaml this the following command : **ros2 param dump /turtlesim > turtlesim.yaml**.

Commands to copy the .yaml file in a host folder in order to read it with gedit.
docker cp container_ID_or_container_name : file_name path_to_the_host_folder.

```
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~$ docker cp limo_navigation:turtlesim.yaml /home/kelly/XDU_internship.github.io/
Successfully copied 2.05kB to /home/kelly/XDU_internship.github.io/
```

To do the inverse operation we do :
docker cp path_to_the_host_folder: file_name container_ID_or_container_name.

Result :

```
/turtlesim:
  ros__parameters:
    background_b: 255
    background_g: 255
    background_r: 69
    qos_overrides:
      /parameter_events:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last
          reliability: reliable
    use_sim_time: false

```

We can also load the .yaml file with the following commands :
ros2 param load /turtlesim turtlesim.yaml

Load a parameter on startup

This defines the configuration and behavior of the system before any actually starts running. It can help to create an environment for a robot, and as soon as we run the node, the window appears as we defined in the parameter loading on startup.

ACTIONS

As in the services, we have servers and clients, but here we have a goal , feedback and result.

1-To see all the actions : ros2 action list

```
root@593aa2fb8316:/# ros2 action list
/turtle1/rotate_absolute
```

to list with type we add -t at the end of the previous command.

```
root@593aa2fb8316:/# ros2 action list -t
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

2- to get information about an action : ros2 action info action_name

```
root@593aa2fb8316:/# ros2 action info /turtle1/rotate_absolute
Action: /turtle1/rotate_absolute
Action clients: 2
    /teleop_turtle
    /teleop_turtle
Action servers: 2
    /turtlesim
    /turtlesim
root@593aa2fb8316:/#
```

3- To see the interface of communication between the node (sever and client) :
ros2 interface show turtlesim/action/action_name

```
root@593aa2fb8316:/# ros2 interface show turtlesim/action/RotateA
bsolute
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

4- to send an action goal :
ros2 action <action_name> <action_type> <goal>.
Eg : ros2 action /turtle1/rotate_absolute turtlesim/action/RotateAbsolute ‘{theta : 1.57}’.
Result : the turtle rotate.

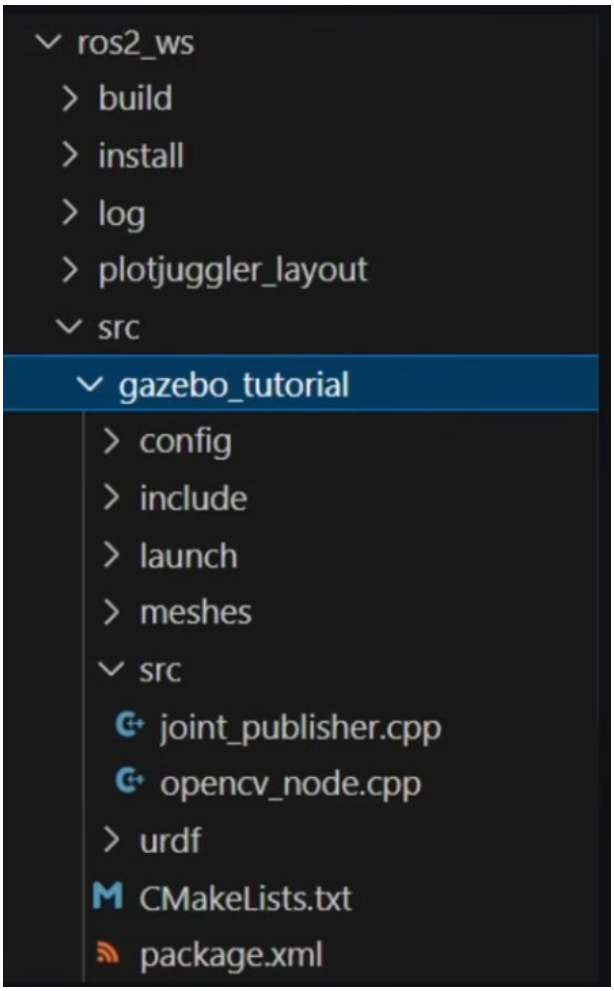
Current problem :
Sometime some process are running in background without let us know, but they presence appears in the listing of the current processes. To see what processes are running in background we can enter the command : ps aux | grep teleop , we will see something like the following image :

```
root@593aa2fb8316:/# ps aux | grep teleop
root          92  0.0  0.1  35040 23552 pts/1    S+   06:04   0:00 /usr/bin/python3 /opt/ros/humble/bin/ros2 run turtle
sim turtle_teleop_key
root          93  0.1  0.1 646364 23572 pts/1    Sl+  06:04   0:05 /opt/ros/humble/lib/turtlesim/turtle_teleop_key
root        4811  0.0  0.1  35040 23524 pts/5    T    06:56   0:00 /usr/bin/python3 /opt/ros/humble/bin/ros2 run turtle
sim turtle_teleop_key
root        4812  0.0  0.1 647136 23652 pts/5    Tl   06:56   0:02 /opt/ros/humble/lib/turtlesim/turtle_teleop_key
root        4969  0.0  0.0   4024  2080 pts/6    S+   07:38   0:00 grep --color=auto teleop
```

When a program starts even into Docker, it is assigned a unique tracking number by the operating system. The number are assigned with the time, and we use the number to stop the running processes.
To stop them we can use :
1- kill 92 93 4811 4969 : which asks the program to stop, and it's clean and safe.
2-kill -9 92 93 4811 4969 : which forces the kernel to stop, and it's fast and dangerous.

« ps aux » display all the programs, but to see those related to teleop we put « **| grep teleop ».**

ROS WORKSPACE



In the picture above, we have a folder that is the workspace named ros2_ws, and it contains 5 main folders : **build, install , log, plotjuggler_layout and src.**
It is into src we will put our openCV codes and build them.

COLCON BUILD TOOL

It's the command used to build packages. In the following , we will write our own code, and it's with colcon that we will build them.

Colcon commands to build packages

1- cloning the ros_tutorials repository.
git clone https://github.com/ros/ros_tutorials.git -b humble

error :

```
root@593aa2fb8316:~/ros2_ws/src# git clone https://github.com/ros/ros_tutorials.git -b humble
Cloning into 'ros_tutorials'...
fatal: unable to access 'https://github.com/ros/ros_tutorials.git/': Could not resolve host: github.co
m
```

The cloning failed because the internet lacks in Docker, so we should run Docker again but this time we should give him the access to internet, and we do it with the following command :
docker run -it --network=host <your_image_name_or_id> bash

/!\ Unfortunately, we should do it when we are creating the container the first time.

But we can clone the repository from the host terminal by indicating the path of the « folder src » of the container.
Command :
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~$ cd /home/kelly/ros2_ws/src
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~/ros2_ws/src$
git clone https://github.com/ros/ros_tutorials.git -b humble.

```
root@593aa2fb8316:~/ros2_ws/src# ls
limo_ros2  ros_tutorials
```

2- Setup colcon tab completion
Colcon tab completion is a terminal shortcut features that saves us from manual typing out long packages names, build path, or command arguments.
We do the setup with :
echo "source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc

3- Build with colcon from ~/ros2_ws
The –symlink-install creates symbol links (symlink) to the original files to the source and builds directories instead of making copies.

Command : colcon build –symlink-install

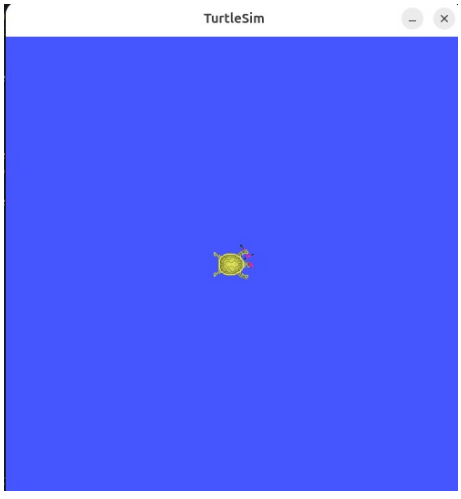
I built with Colcon, and at the end of the building, it is mentioned « 5 packages finished », but when I do « ros2 pkg list », I see packages more than 5. The confusion is that ros2 pkg list list all the packages on the computer, and the 5 packages mentioned at the end of the building are in the

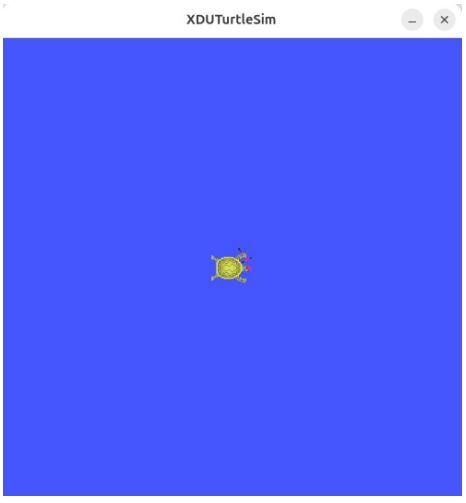
workspace ros2_ws.

```
root@593aa2fb8316:~/ros2_ws/install# cd ..
root@593aa2fb8316:~/ros2_ws# colcon build --symlink-install
Starting >>> limo_msgs
Starting >>> turtlesim
Starting >>> limo_car
Starting >>> limo_description
Finished <<< limo_description [0.35s]
Finished <<< limo_car [0.38s]
Finished <<< limo_msgs [0.96s]
Finished <<< turtlesim [26.3s]
Starting >>> limo_base
Finished <<< limo_base [0.15s]

Summary: 5 packages finished [27.0s]
```

4- Sourcing overlay from ros_ws
This allows to run the packages we have built.
Command : source install/setup.bash

5- Verifying we are modifying our file and not the previous one by editing the windows
The frame that forms the environment on simulation has a parameter at the line 52 that allows to give a name to the current windows.
command : setWindowTitle(‘’ XDUTurtleSim ‘’)





CREATE PACKAGES

1- Create packages :
We create the new packages into src.
Command :
ros2 pkg create --build-type ament_cmake --node-name <node_name> <package_name>

Result :

```
root@593aa2fb8316:~/ros2_ws/src# ros2 pkg create --build-type ament_cmake --node-name my_node my_package
going to create a new package
package name: my_package
destination directory: /root/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['root <kassingit@gmail.com>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: []
node_name: my_node
creating folder ./my_package
creating ./my_package/package.xml
creating source and include folder
creating folder ./my_package/src
creating folder ./my_package/include/my_package
creating ./my_package/CMakeLists.txt
creating ./my_package/src/my_node.cpp

[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.xml, but no LICENSE fil
e has been created.
It is recommended to use one of the ament license identitifers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
```

2- To build my package :
command : colcon build –packages-select my_package

PUBLISHER AND SUBSCRIBER CREATION (C++)

1- Create the package pubsub (to refer to Publisher and Subscriber) :
ros2 pkg create --build-type ament_cmake cpp_pubsub

2- Update the package.xml and CmakeLists.txt
We must add some codes in the file package.xml and CmakeLists.txt
(refer to part added)

3- Checking for missing dependencies (from ros2_ws)
command : rosdep install -i --from-path src --rosdistro humble -y

4- Build cpp_pubsub
Command : colcon build –packages-select cpp_pubsub

5- run the publisher
source install/setup.bash
ros2 run cpp_pubsub talker

6- run the subscriber
source install/local_setup.bash
ros2 run cpp_pubsub listener

PUBLISHER AND SUBSCRIBER CREATION (Python)

1- Create the package pubsub (to refer to Publisher and Subscriber) :
ros2 pkg create --build-type ament_python cpp_pubsub

Note : for creating a package with python we just replace cmake by python.

2- Update the package.xml and setup.py
We must add some codes in the file package.xml and setup.py
(refer to part added)

3- Checking for missing dependencies (from ros2_ws)
rosdep install -i --from-path src --rosdistro humble -y

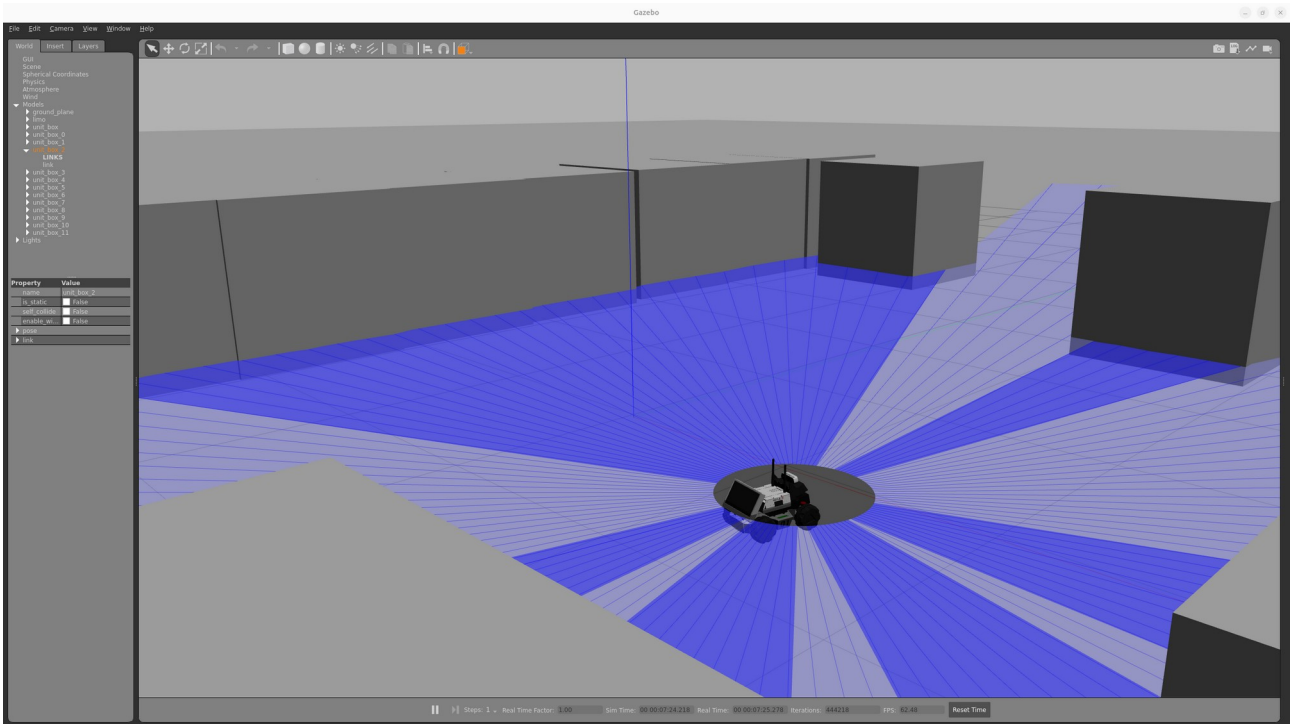
4- Build cpp_pubsub
colcon build --packages-select cpp_pubsub

5- run the publisher
source install/local_setup.bash
ros2 run py_pubsub talker

6- run the subscriber
source install/setup.bash
ros2 run py_pubsub listener

GAZEBO

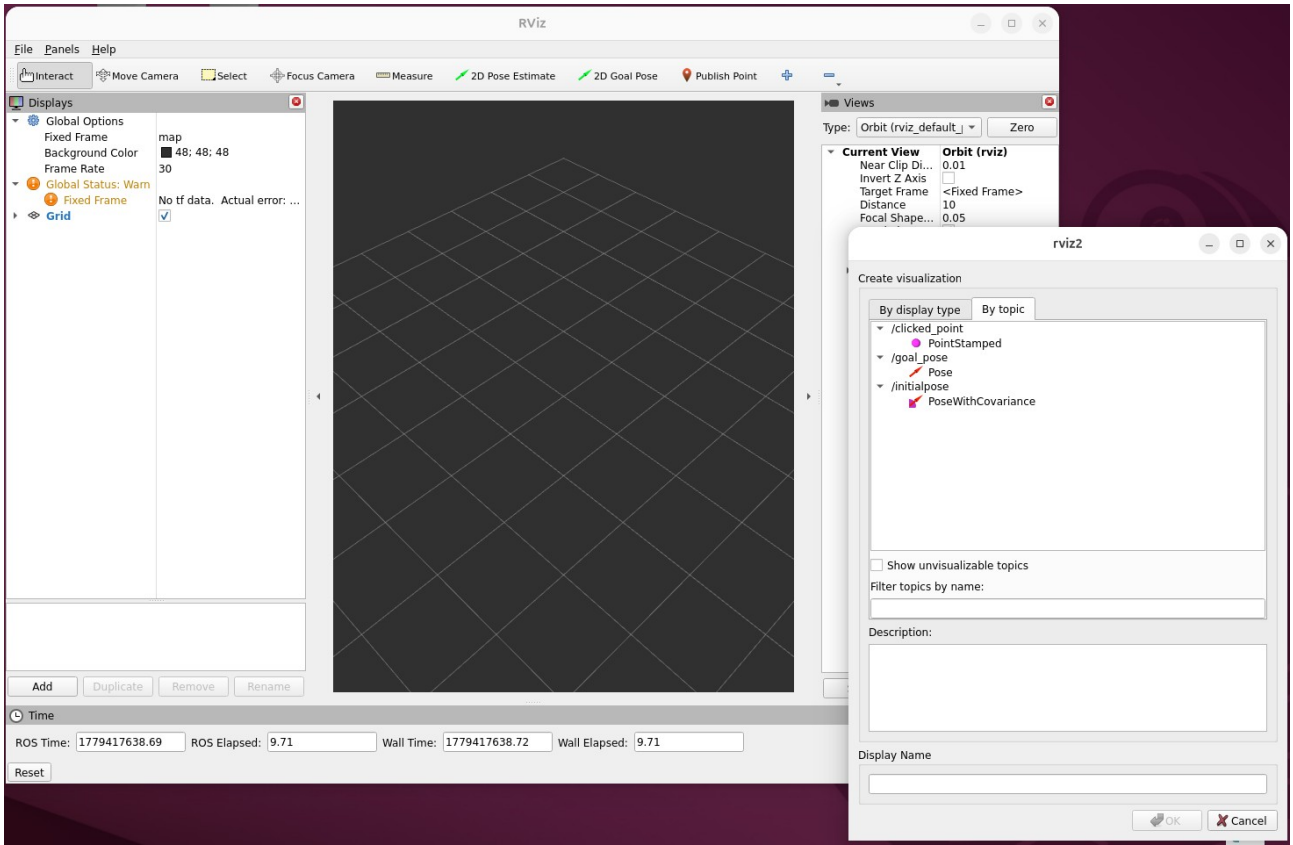
I have finally succed to install Gazebo and configure it. Now it is stable on my computer for simulation. Below is a picture.



This version is fast and doesn’t bug at all.

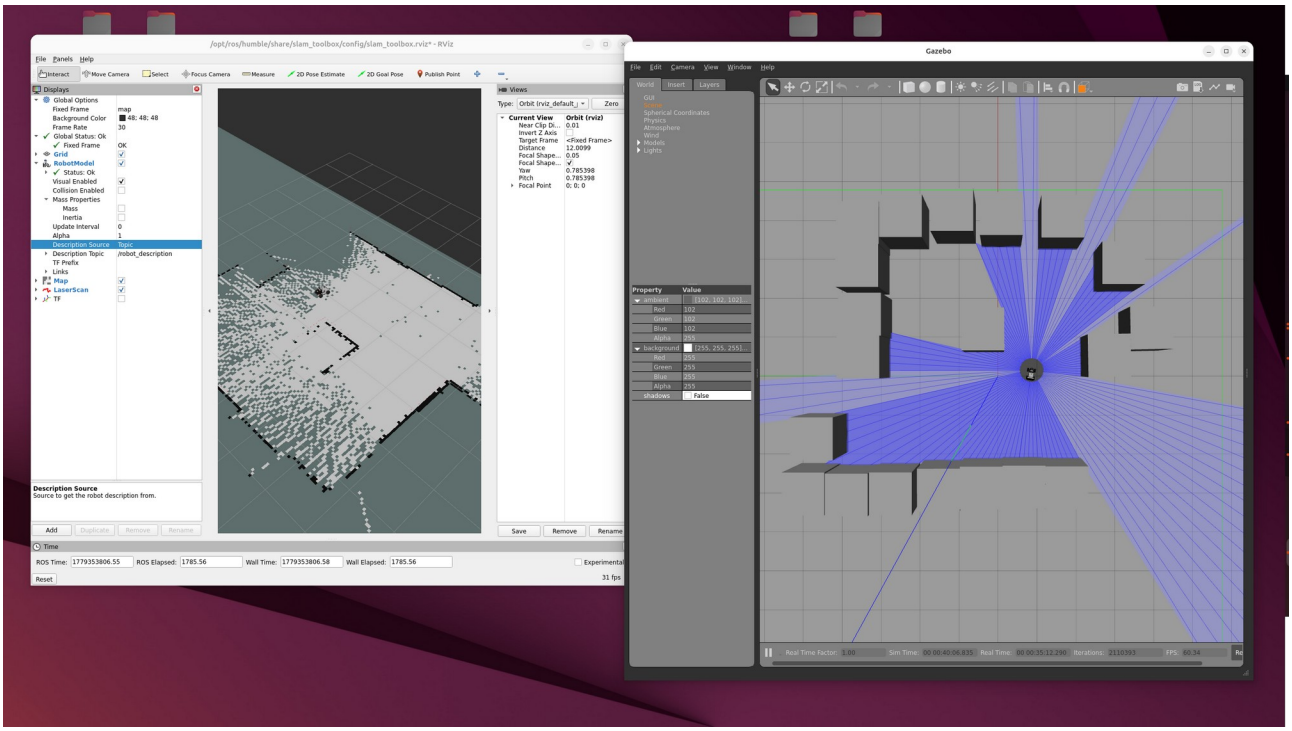
RVIZ

To visualize the map while it is creating Rviz is useful. It will serve to create the maps and implement the robot navigation.



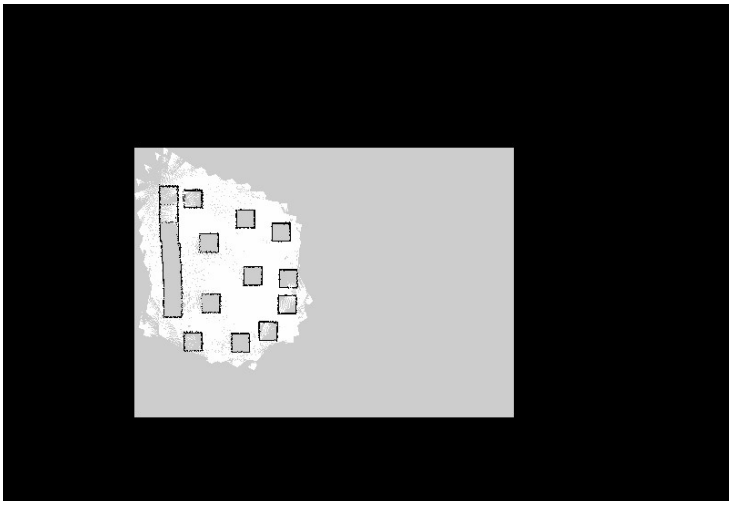
SLAM

Thanks to the toolbox of Slam, Slam use is not difficult. Slam helps us to create a map for the car. To do it, we

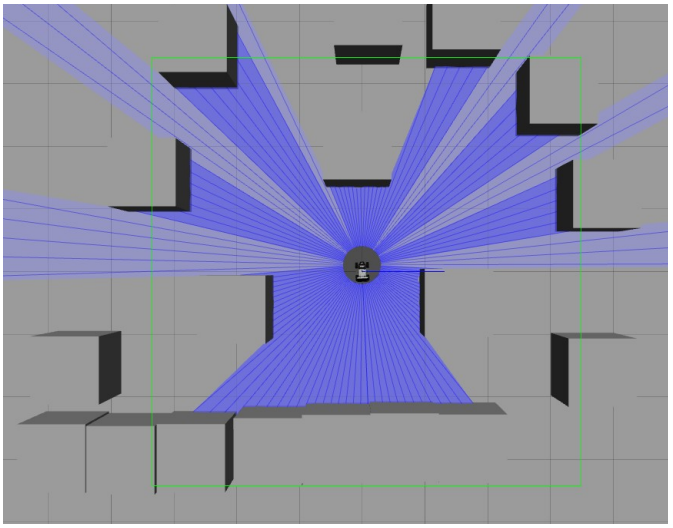


Thanks to it, I have succed to generate a maps for the robot. This map will be used for the autonomous navigation later. Below is the map.

Maps created with Slam

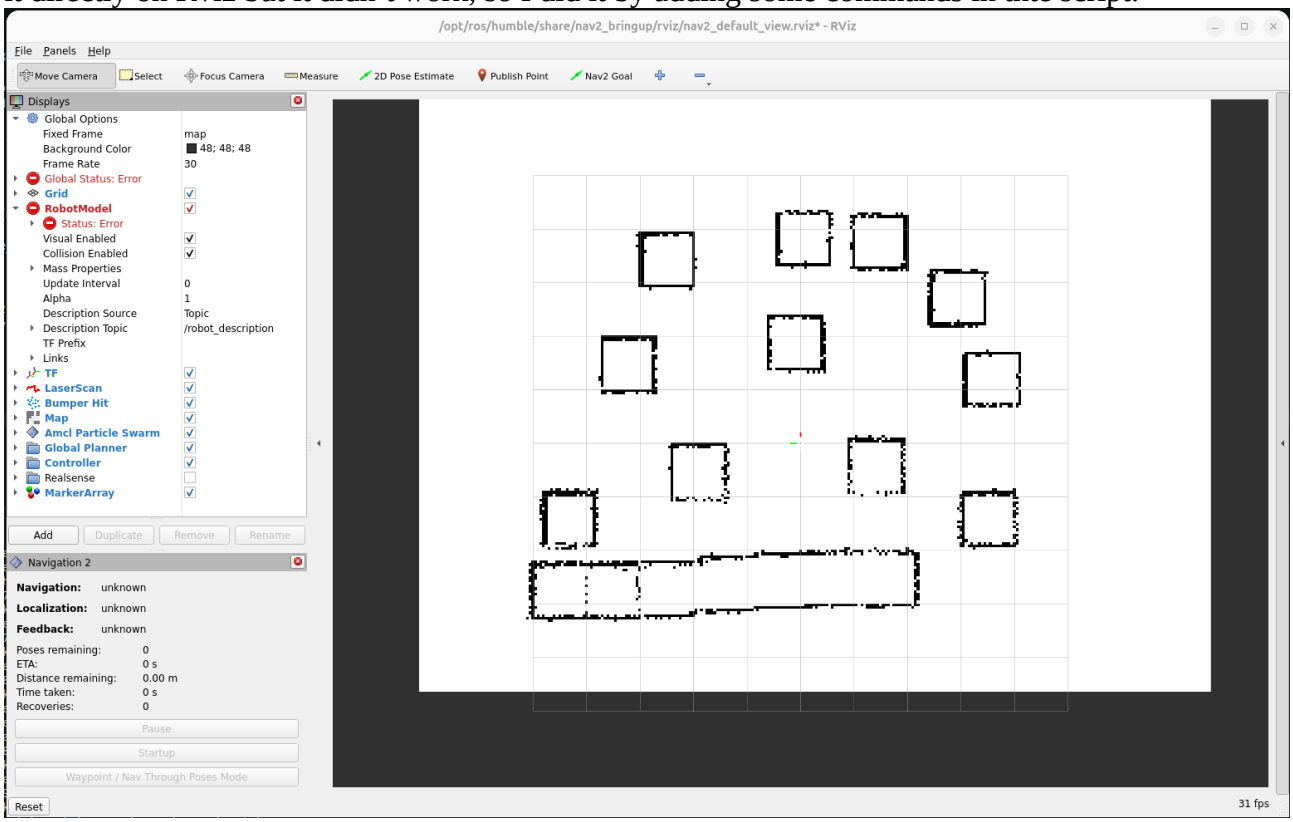


Environment on Gazebo



AUTONOMOUS NAVIGATION

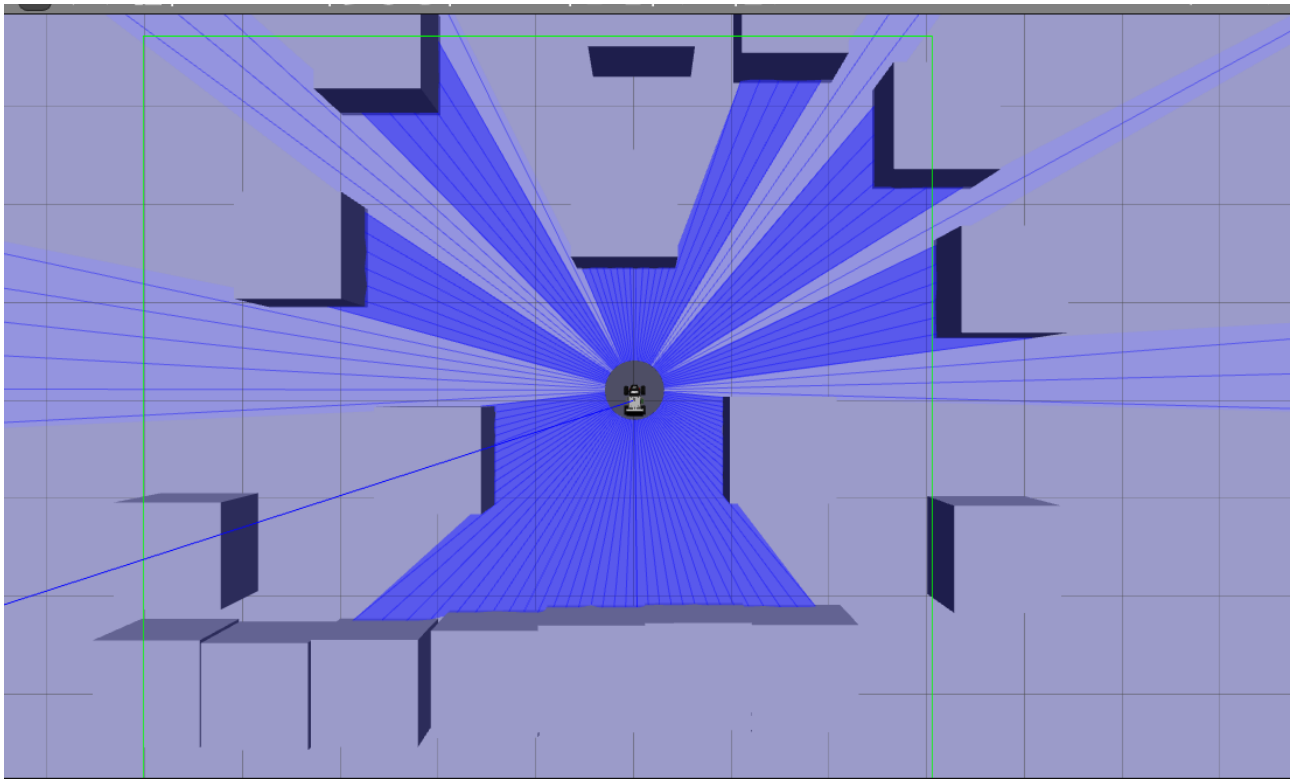
The map didn’t display before , the problem was that the origin of … should be given, I tried to do it directly on Rviz but it didn’t work, so I did it by adding some commands in thte script.

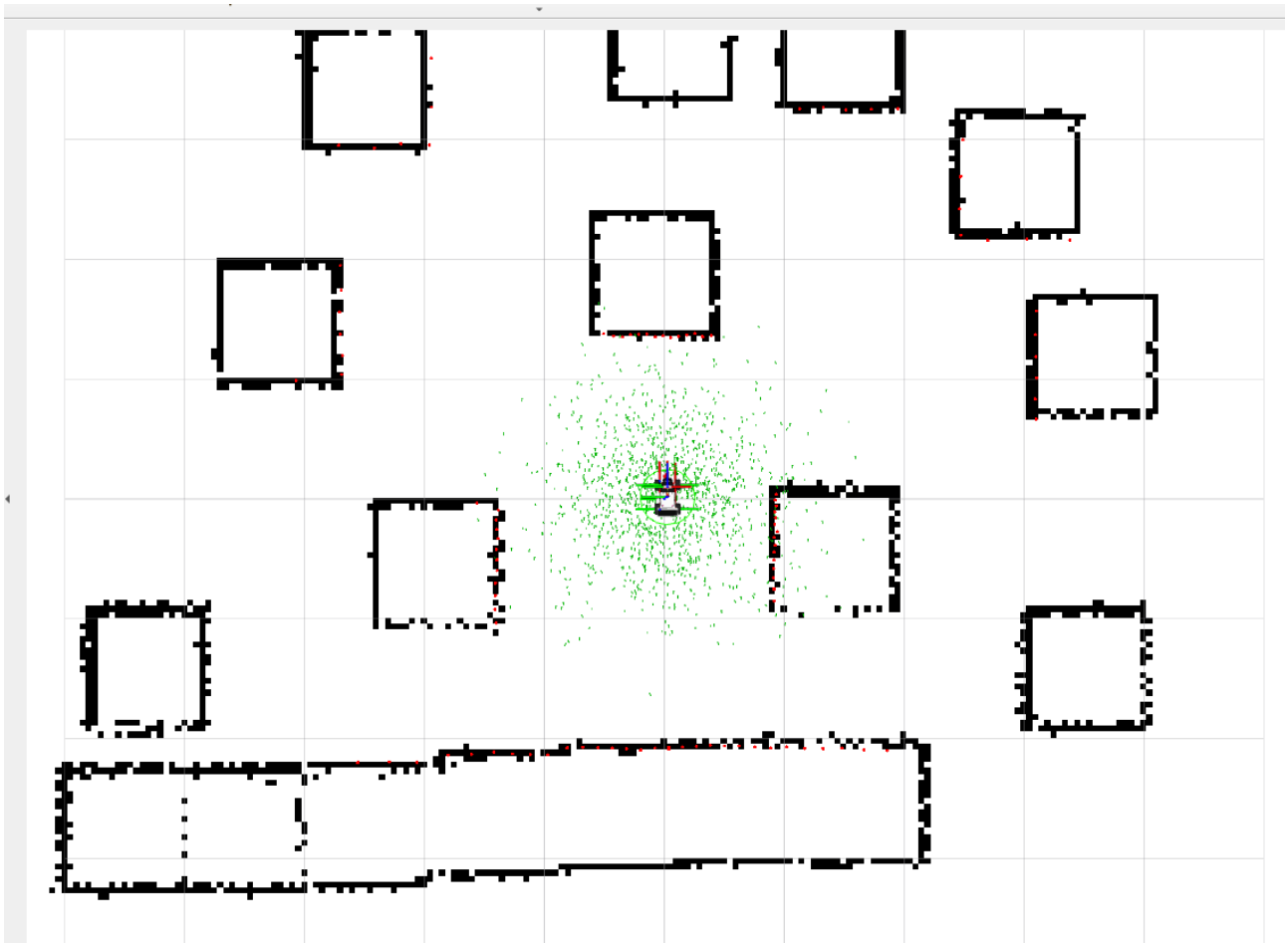


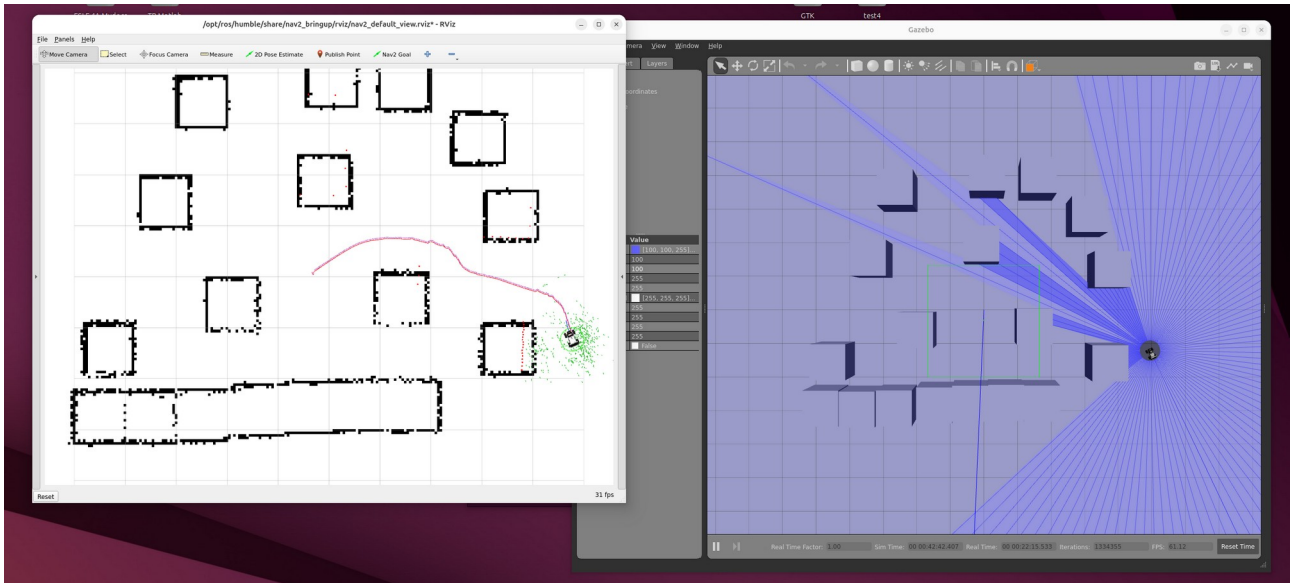
The map displays but the is still the following message :

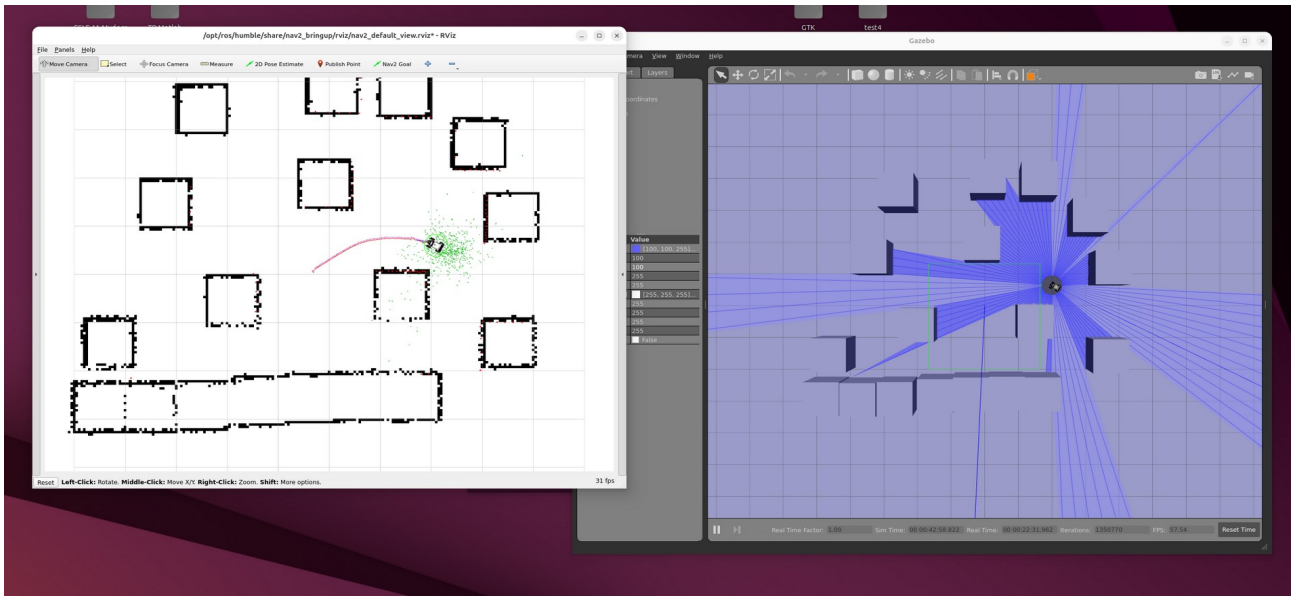
[component_container_isolated-1] [INFO] [1779522901.987827325] [global_costmap.global_costmap]: Timed out waiting for transform from base_link to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[component_container_isolated-1] [INFO] [1779522902.487829974] [global_costmap.global_costmap]: Timed out waiting for transform from base_link to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[component_container_isolated-1] [INFO] [1779522902.987827796] [global_costmap.global_costmap]: Timed out waiting for transform from base_link to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[component_container_isolated-1] [INFO] [1779522903.487836000] [global_costmap.global_costmap]: Timed out waiting for transform from base_link to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist

After configuring Rviz to display the map, we carte see the map created and display near the original environment.

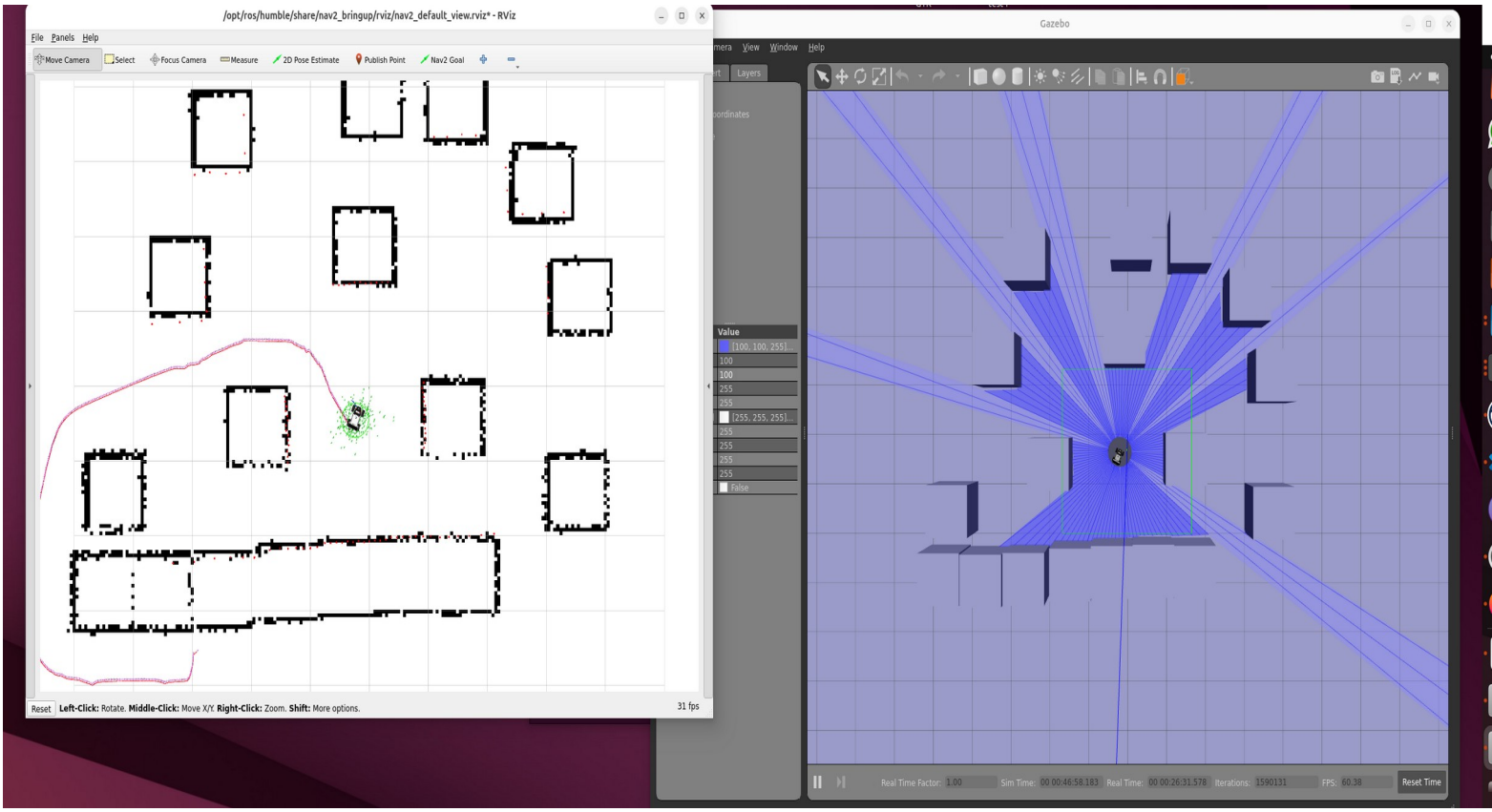




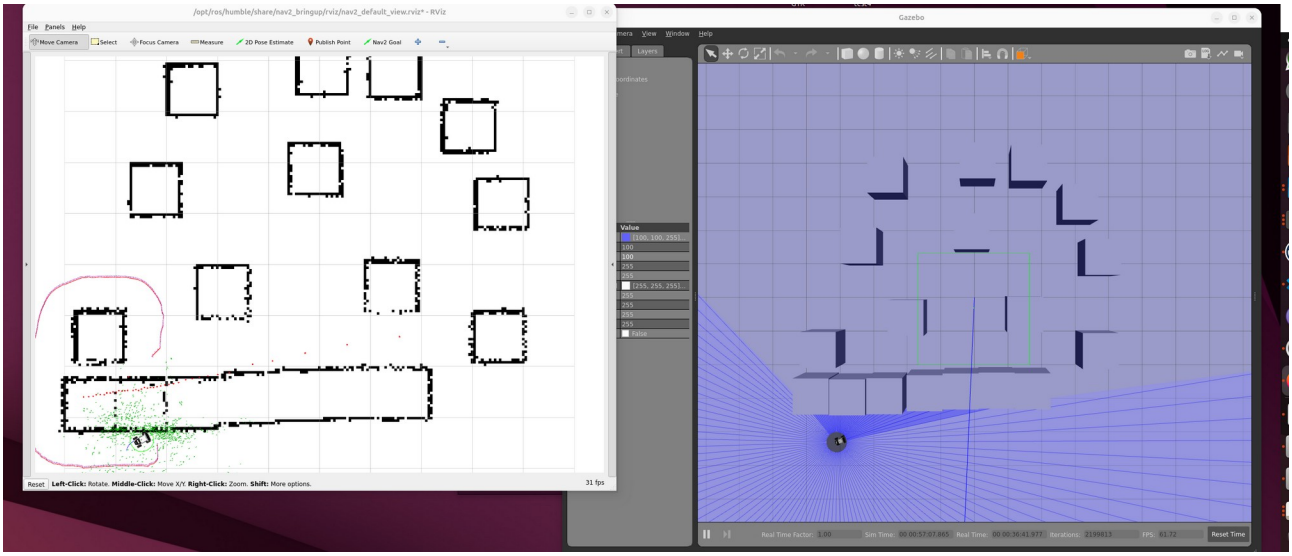

/opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz* - RViz
File Panels Help
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
Publish Point
Nav2 Goal
Reset
31 fps
Gazebo
Edit View Window Help
Layers
Value
Real Time Factor: 1.00
Sim Time: 00 00:42:42.407
Real Time: 00 00:22:15.531
Iterations: 1334355
FPS: 61.12
Reset Time


/opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz* - RViz
File Panels Help
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
Publish Point
Nav2 Goal
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click: Zoom. Shift: More options.
31 fps
Gazebo
Edit View Window Help
Layers
Value
Real Time Factor: 1.00
Sim Time: 00 00:42:58.822
Real Time: 00 00:22:31.962
Iterations: 1350770
FPS: 57.54
Reset Time

There is one problem, the car doesn’t take the short path to go from one point to another like on this picture.



Another problem.
The algorithm behing the navigation isn’t optimal. In the picture below, in Gazebo (at right) the car has wide place in front of it, but in Rviz, it thinks that it is going straight in a wall, and this has created the stop of the robot.



This happened because the limit defined by Rvis at the left are the real as in Gazebo. We have to optimize the algorithm or find another solution to solve this problem.

AUTONOMOUS NAVIGATION

Remote Desktop Access

As the all the software and all useful data are on the Limo car, to use them we decided to display the screen of the car on my computer, which is called « remote desktop access ». To do so, we use the software Remmina directly integrate in my Ubuntu. Remmina is a screen share software.
Here are the step needed to do it :

1- Know the VNC (Virtual Network Computing) server of the car and activate it

-Activation of the VNC : vncserver -geometry 1280x720 :1
-The type of VNC server :
dpkg -l | grep -E "vnc|vino|x11vnc"

Result :
agilex@master:~$ dpkg -l | grep -E "vnc|vino|x11vnc"
ii libvncclient1:arm64 0.9.12+dfsg-9ubuntu0.3 arm64
API to write one's own VNC server - client library
ii remmina-plugin-vnc:arm64 1.4.2+dfsg-1ubuntu1 arm64
VNC plugin for Remmina
ii vino 3.22.0-5ubuntu2.2 arm64
VNC server for GNOME

This show that there is a VNC named Vino on the Limo car.

-To awake Vino :
export DISPLAY=:0
/usr/lib/vino/vino-server --display=:0 &

-**To verify if vino is listening :** sudo ss -tlnp | grep 5900 (5900 for the port 5900, which a classique port used)

we must see something like this :
agilex@master:~/limo_ros2_ws/src/limo_ros2/limo_bringup/maps$ sudo ss -tlnp | grep 5900
[sudo] password for agilex:
LISTEN 0 5 0.0.0.0:5900 0.0.0.0:* users:(("vino-server",pid=45463,fd=12))
LISTEN 0 5 [::]:5900 [::]:* users:(("vino-server",pid=45463,fd=11))

Something the Remmina screen can be black, and it is because the car can being running Vino, ToDeskt and NoMachine which are three software for the remote desktop access. To use Vino we have to shutdwon the others. To do it we use :

1. **To shutdown ToDesk and NoMachine**

sudo systemctl stop todeskd 2>/dev/null
sudo /etc/init.d/nxserver stop 2>/dev/null

2. To give the permission to acces to the local screen :0

export DISPLAY=:0
xhost +local:agilex 2>/dev/null

3. To force Vino to accept access without password

gsettings set org.gnome.Vino require-encryption false
gsettings set org.gnome.Vino prompt-enabled false
gsettings set org.gnome.Vino authentication-methods "['none']"

4. To un Vino again in front task

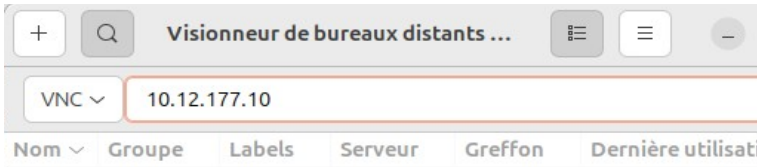
killall vino-server 2>/dev/null
/usr/lib/vino/vino-server --display=:0 &

N: if want to acces with a password we can do it, as following.
- gsettings set org.gnome.Vino authentication-methods "['vnc']"
- echo -n 'agilex' | base64
The terminal will send a string like YWdpbGV4 that we use like this :
-gsettings set org.gnome.Vino vnc-password 'YWdpbGV4'

2-Know the Ip address of the car

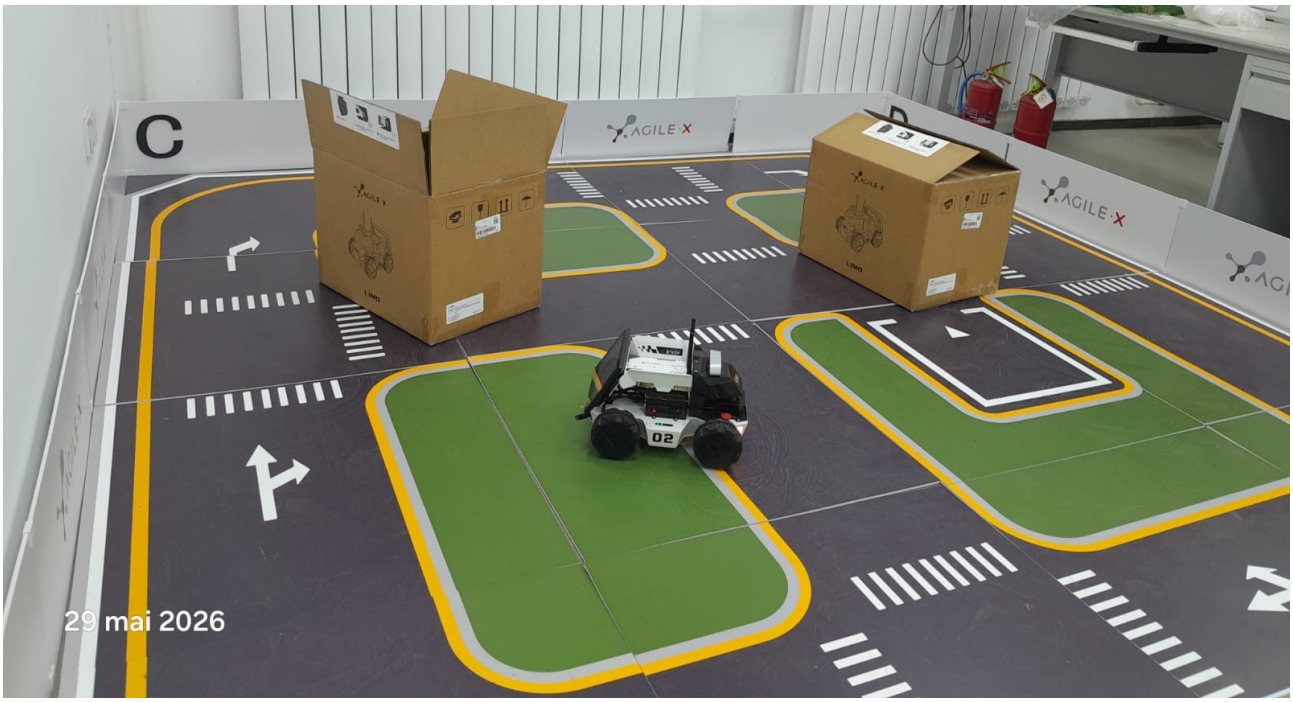
10.12.177.10

3- Open Remmina and connect ourself to the car



LAUNCH FILE FOR AUTONOMOUS NAVIGATION

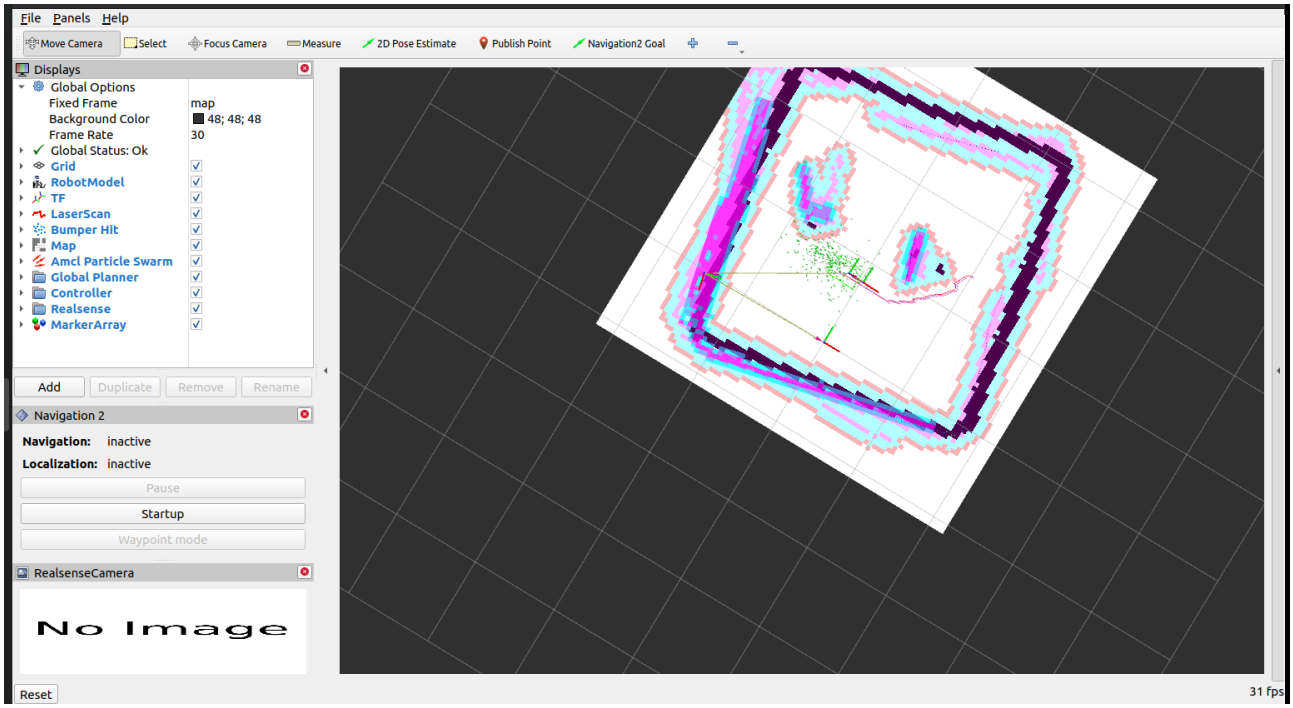
To do the autonomous navigation, we made a model of field with only 2 obstacles, as on the following picture :

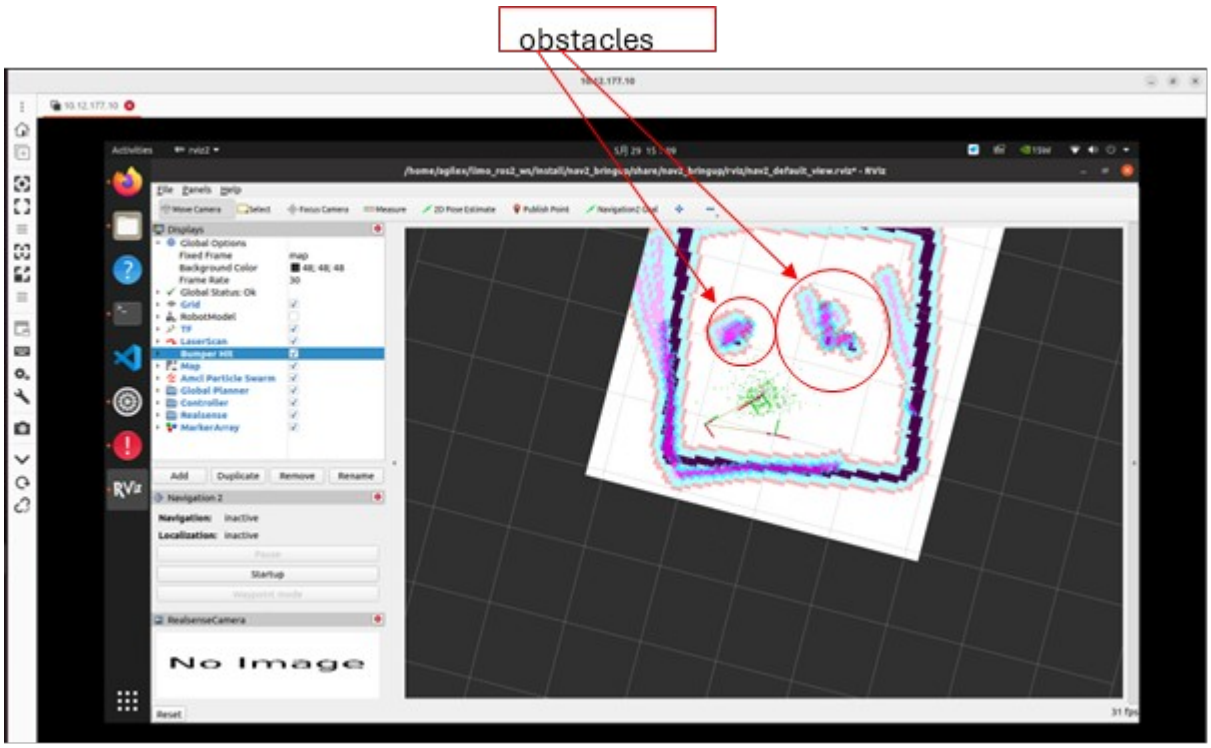


We configure the file **limo_nav2.launch.py** which is our launch file, its path is **/home/agilex/limo_ros2_ws/src/limo_ros2/limo_bringup/launch/limo_nav2.launch.py.**

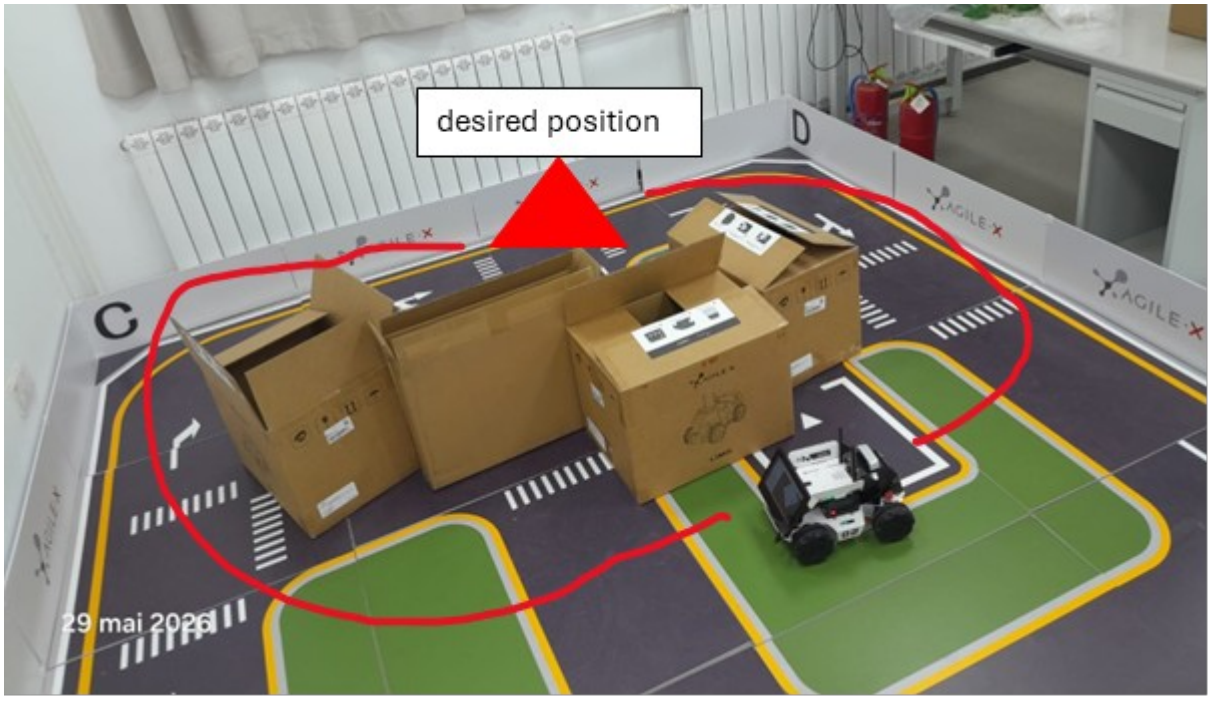
```python
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    limo_bringup_dir = get_package_share_directory('limo_bringup')
    nav2_bringup_dir = get_package_share_directory('nav2_bringup')

    use_sim_time = LaunchConfiguration('use_sim_time', default='false')
    map_yaml_path = LaunchConfiguration('map',default=os.path.join(limo_bringup_dir,'maps','map_test1.yaml'))
    nav2_param_path = LaunchConfiguration('params_file',default=os.path.join(limo_bringup_dir,'param','nav2.yaml'))

    rviz_config_dir = os.path.join(nav2_bringup_dir,'rviz','nav2_default_view.rviz')

    return LaunchDescription([
        DeclareLaunchArgument('use_sim_time',default_value=use_sim_time,description='Use simulation (Gazebo) clock if true'),
        DeclareLaunchArgument('map',default_value=map_yaml_path,description='Full path to map file to load'),
        DeclareLaunchArgument('params_file',default_value=nav2_param_path,description='Full path to param file to load'),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource([nav2_bringup_dir,'/launch','/bringup_launch.py']),
            launch_arguments={
                'map': map_yaml_path,
                'use_sim_time': use_sim_time,
                'params_file': nav2_param_path}.items(),
        ),

        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            arguments=['-d', rviz_config_dir],
            parameters=[{'use_sim_time': use_sim_time}],
            output='screen'),
    ])
```

When we launch this launch file have the follow below.





When a position is indicated the car move and goes there avoiding the obstacles.
So far, we succed to make a autonomous navigation, but there a some problem, the car has difficulties with some new obstacles like on the picture below.



Next time, we plan to work on the efficiency of obstacle avoidance and finding the best way to reach the desired position.

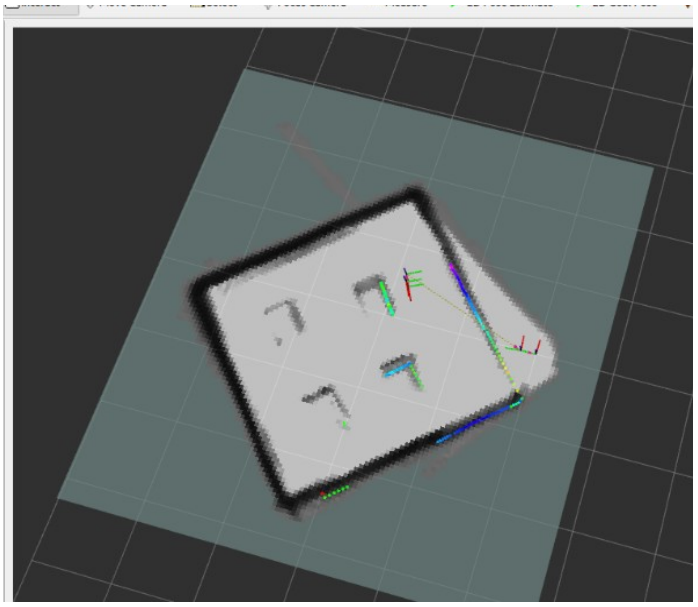
Improvement :

1- To make a map with 4 obstacle like a field

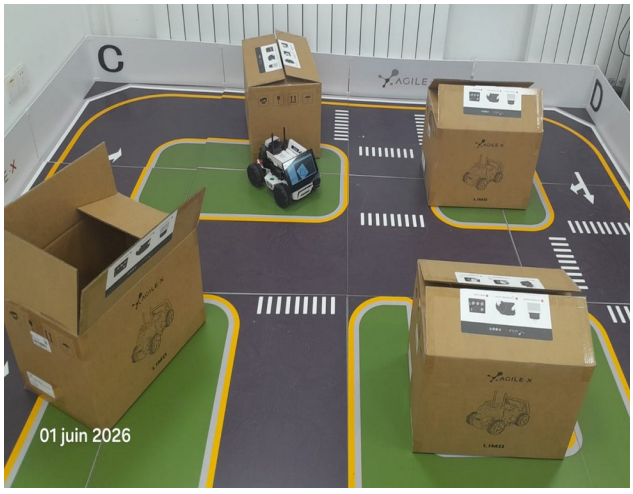
2- Make a autonomous navigation and report the problem encounters like the lack of accurancy.

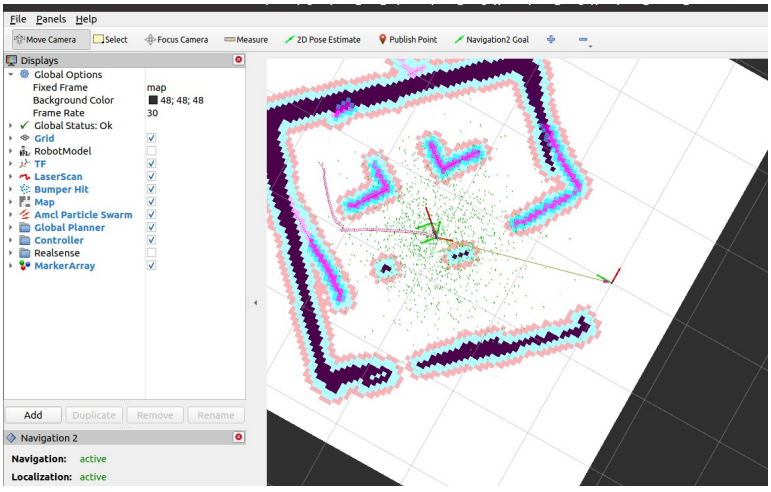
3- Find some way to improve the autonomous navigation.

1- To make a map with 4 obstacle like a field



2- Make a autonomous navigation and report the problem encounters like the lack of accurancy.





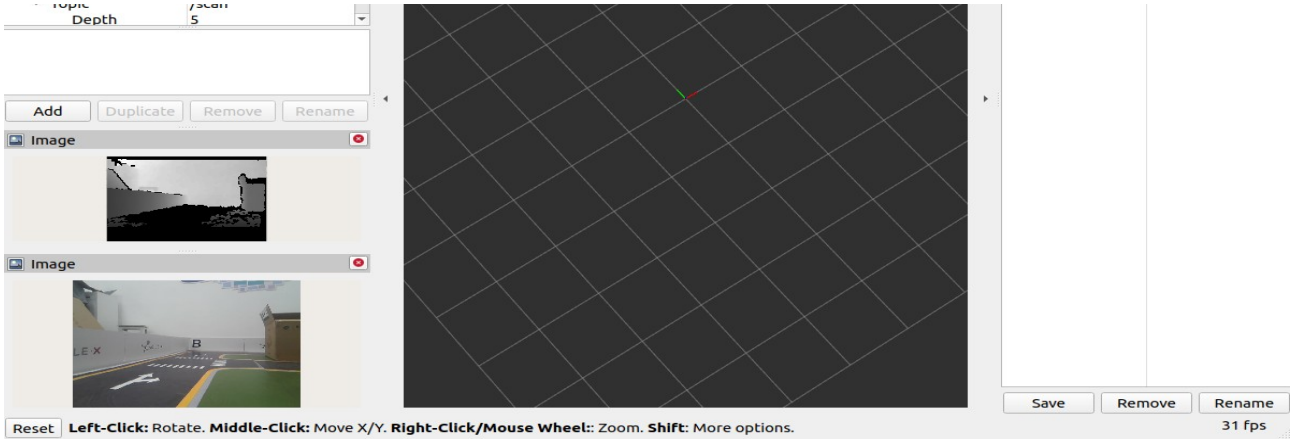
The autonomous navigation works, now we must improve it by adding obstacle avoidance and see if we can make the navigation more quick because the car often takes time to go to the desired position and hits the obstacle on the way.

3- Find some way to improve the autonomous navigation.
- For the obstacle avoidance we will used the ultrasonics sensors.
- To make the car more rapid, we work on the algoritm A* .

Before doing these improvement we will see if there is not avaible features we can use to improve the navigation.

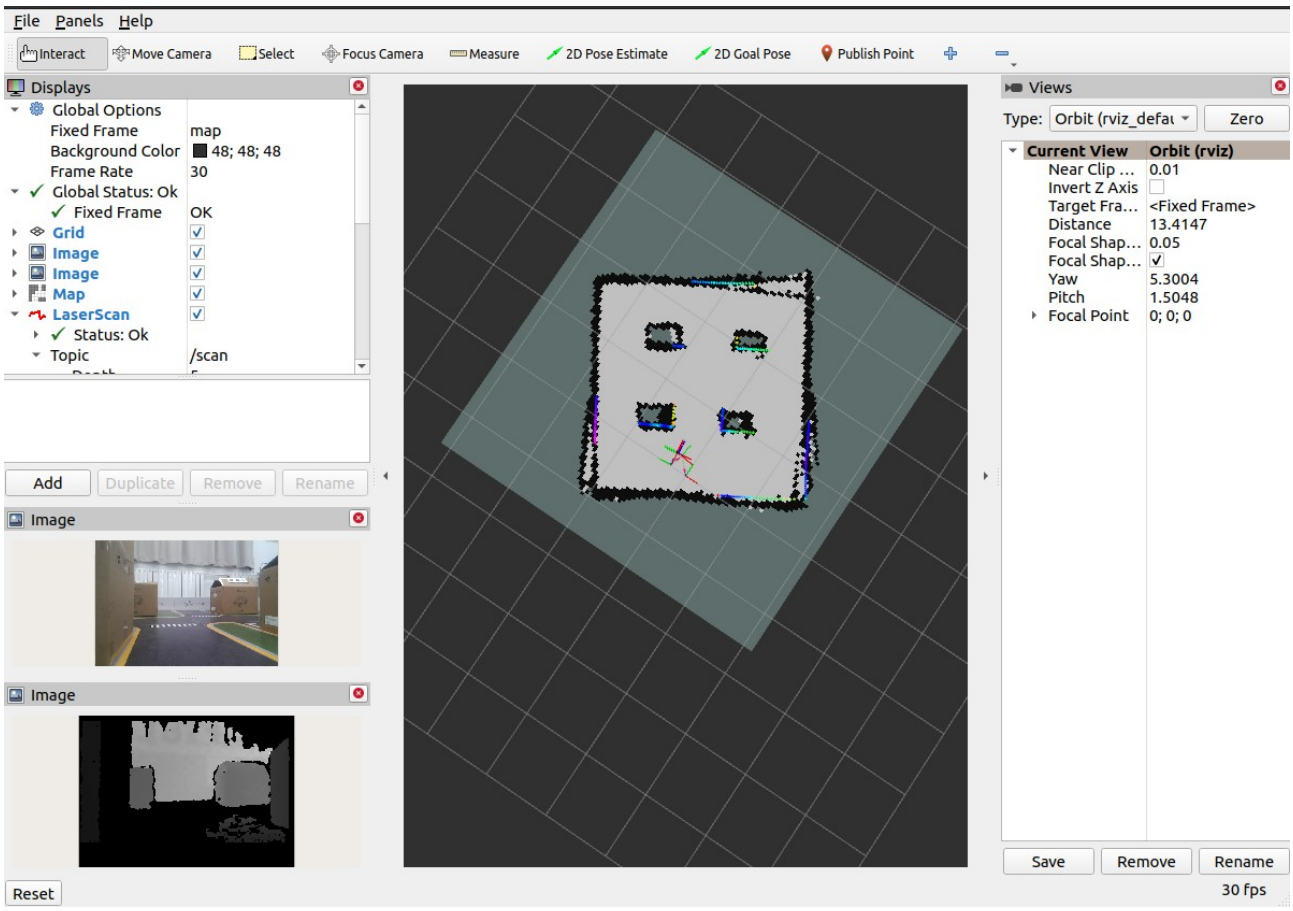
DEPTH CAMERA AND LIDAR MAPPING

1-View depth camera information



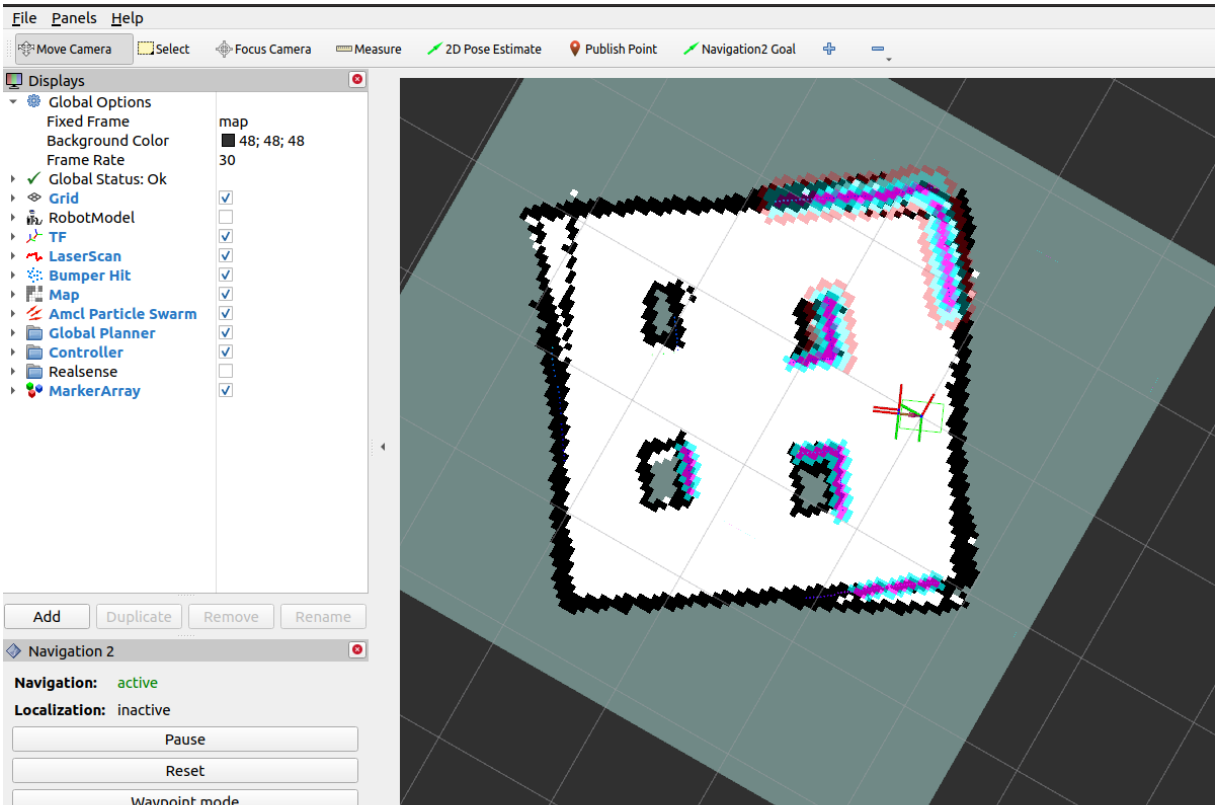
To access the data send by the depth camera I must subscribe (then it will be the pubisher). This will help me to use the data in my own program.

Mapping :



The resultat is more better than with only Lidar.

Here is what we have for the autonomous navigation.
The navigation file :



NAVIGATION
The autonomous navigation is better with Lidar than Lidar and depth camera.

We can see than the boxes have been moved by the car. Autonomous navigation with Lidar and depth camera is not better than the one with only Lidar, then we will continue with only Lidar.

Improvement of the navigation with only Lidar

We open to see the configuration of the file responsible for the navigaton nav2.yaml, and we saw many parameters responsible for the behavior of the car during the navaigation.

```
255
256 planner_server:
257   ros__parameters:
258     expected_planner_frequency: 20.0
259     use_sim_time: False
260     planner_plugins: ["GridBased"]
261     GridBased:
262       plugin: "nav2_navfn_planner/NavfnPlanner"
263       tolerance: 0.5
264       use_astar: false
265       allow_unknown: true
266
```

At the line 264 (in planner_server) of this capture, it is indicated that A* algorithm is not used. By default the algoritm used is Djistra, but A* is better.

```
120       short_circuit_trajectory_evaluation: True
121       stateful: True
122       critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
123       BaseObstacle.scale: 0.02
124       PathAlign.scale: 32.0
125       PathAlign.forward_point_distance: 0.1
126       GoalAlign.scale: 24.0
127       GoalAlign.forward_point_distance: 0.1
128       PathDist.scale: 32.0
129       GoalDist.scale: 24.0
130       RotateToGoal.scale: 32.0
```

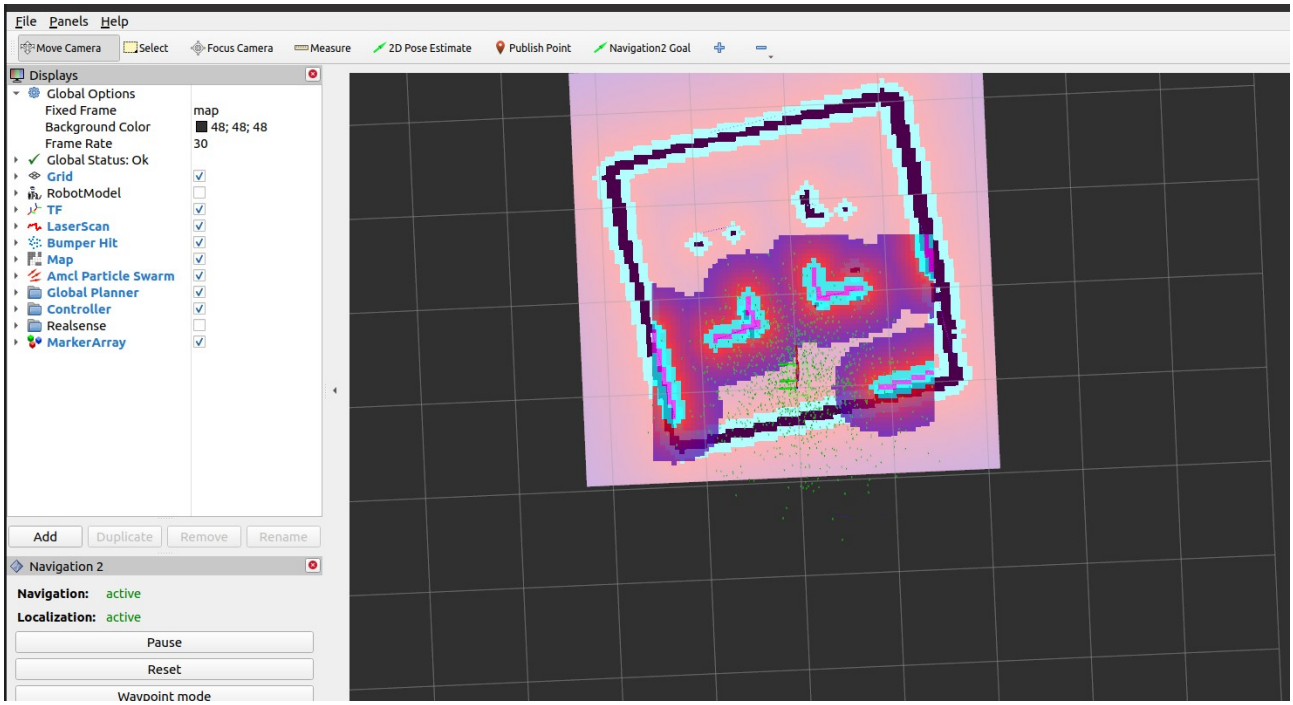
Here the parameter BaseObstacle.scale and PathAlign.scale in controller_server are used to give a weight of prioroty respectively to the obstacles and the path alignement. BaseObstacle.scale is 0.02 (line 123) and PathAlign.scale = 32 (ligne 124), and it means that we prioritize continuing to follow the trajectory even if we must sometime hit obstacles.

```
148       height: 3
149       resolution: 0.05
150       footprint: "[[0.15,0.10],[0.15,-0.10],[-0.15,-0.10],[-0.15,0.10]]"
151       plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
152       inflation_layer:
153         plugin: "nav2_costmap_2d::InflationLayer"
154         inflation_radius: 0.02
155         cost_scaling_factor: 3.0
156       obstacle_layer:
157         plugin: "nav2_costmap_2d::ObstacleLayer"
158         enabled: True
159         observation_sources: scan
```

The inflation_radius is 0.02 (line 154) in local_costmap is the limitation put arround the obstacle to avoid collision between the car and the obstacles, and it is represent by the color bleu, red and purple arroud the obstacle. If its value is high, this limitation meet each other and it can become difficult for the car to move because for it, there is no way. It is the case for the image below where inflation_radius is X only.

To improve the navigation we will change the algorithm used and the parameters inflation_radius PathAlign.scale, and BaseObstacle.scale.

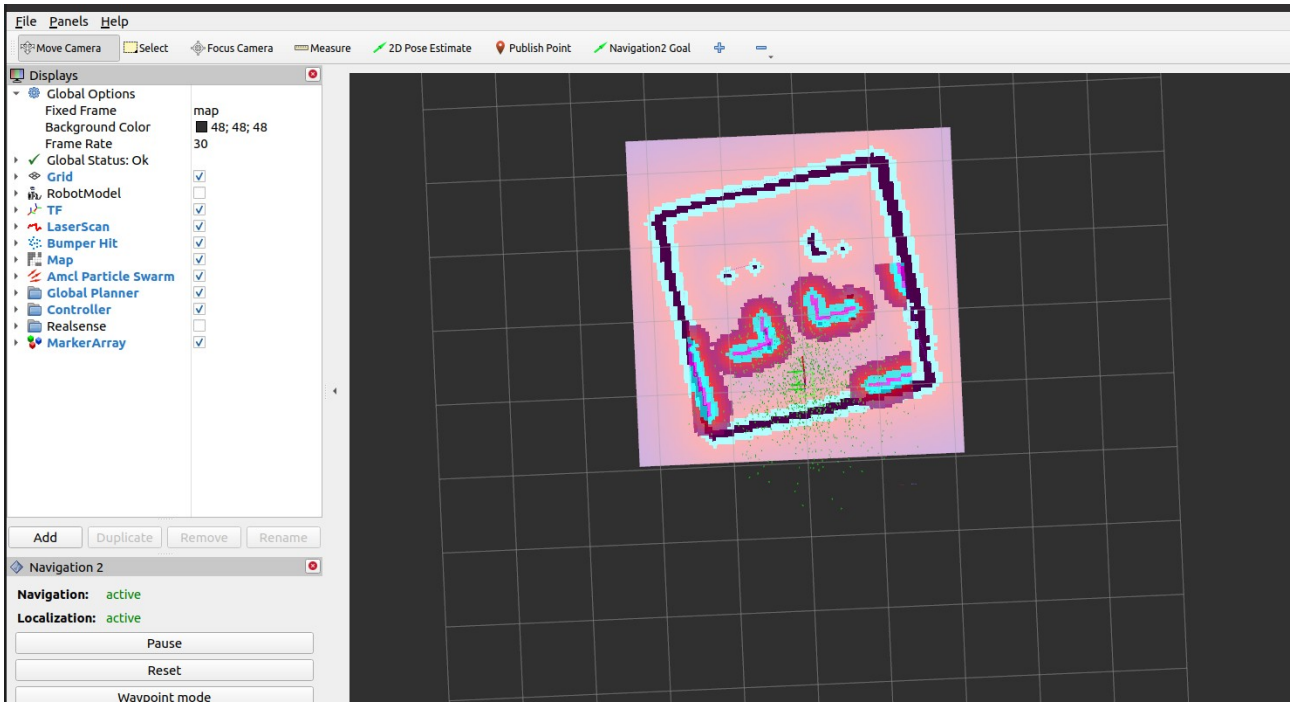
A*	Inflation_radius	BaseObstacle.scale	PathAlign.scale
actived	X	40	32



The result is not satisfaying because the car continue to hit the obstacle and refuse to move sometime to move between to obstacle because of the limitation that met each other.

A*	Inflation_radius	BaseObstacle.scale	PathAlign.scale
actived	0,30	40	32

Result :



The car avoid the obstacle better than above, and the limitation have been reduced. That give to the robot more degree of liberty.

A*	Inflation_radius	BaseObstacle.scale	PathAlign.scale
actived	0,40	34	10

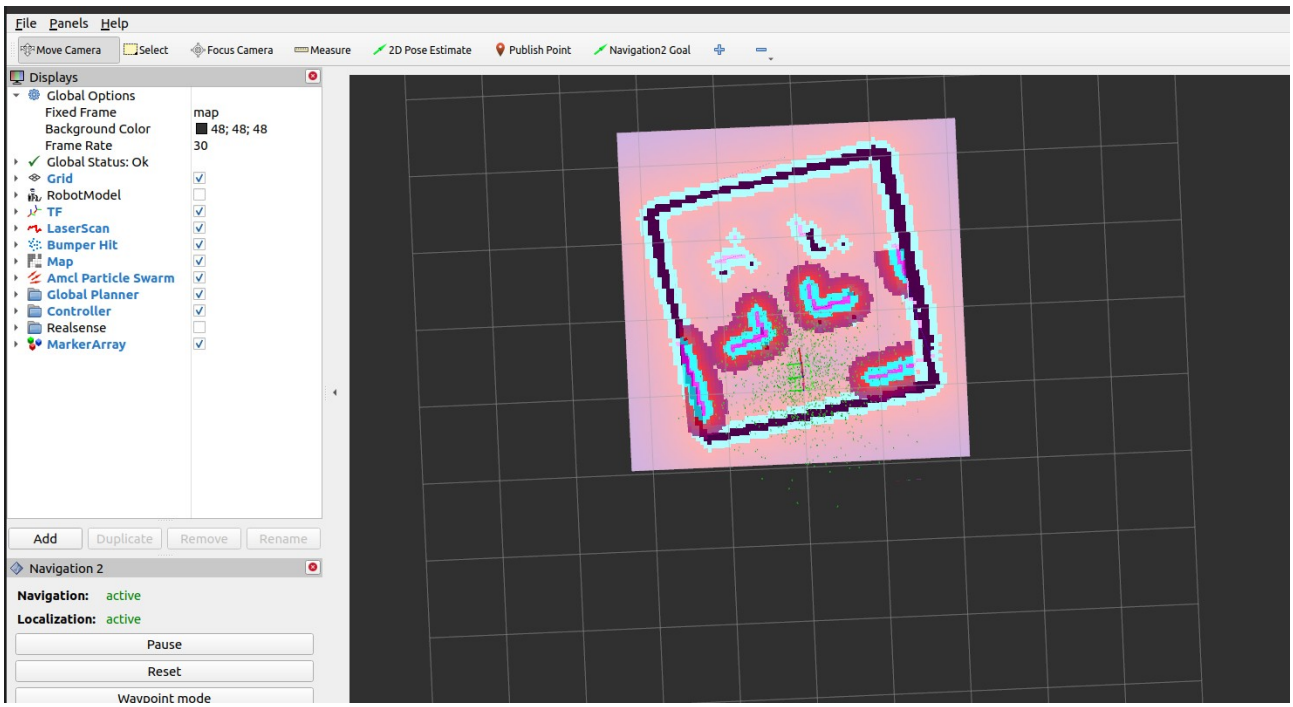
We modified some additional parameters :

PathDist.scale: 10.0 :

PathDist.scale represents la capacity to stay close to the global path.

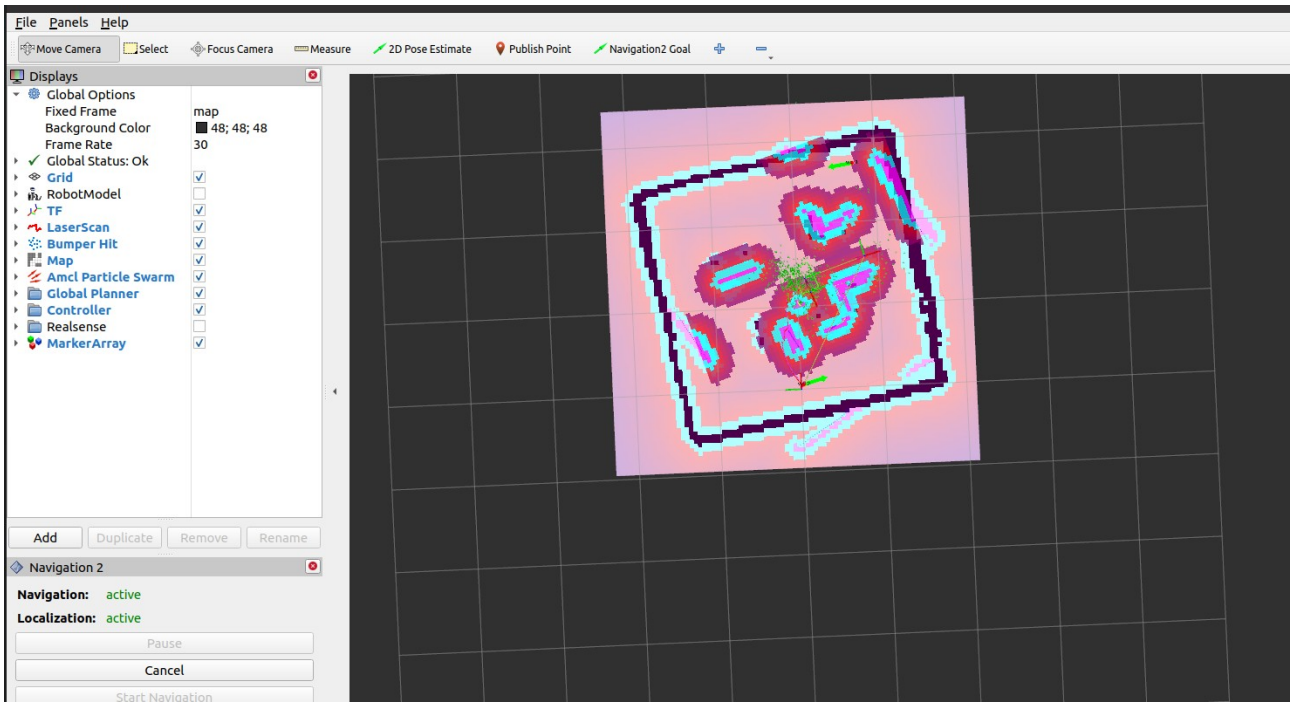
RotateToGoal.lookahead_time: 2.0 :

RotateToGoal.lookahead_time makes the car anticipate the final position, so it slow down when it is close to the final position. This allows the car not to take enough time when it arrived in a place in the desired position.



To how the car can manage itself with new obstacle we put two new obstacles.

The car can not move accros the obstacle because of the limitations that meet each other. If we want to allow the car to pass through tiny space Must reduce the inflation_raduis and increase its importance.



To continue improving the navigation, we will write a new nav2.yaml file and node called **obstacle_avoidance** on which the controller_server will pusblish is data ; once the obstacle avoidance node has received the commands, it will compare them with those coming from the Lidar(/scan) and see if the controller_server commands don’t lead into an obstacle, if yes we don’t send to cmd_vel else we let them go. It is like a filter of commands that will lead in the wall.

To do so, we have created the package **robot_security,** in which we copied-pasted /param/nav2.yaml that we will modify according our need.

Class allowing to create the node : MyNode
Node name : obstacle_avoidance
executable : obstacle_avoidance
file .py : safety_filter.py

```python
#!/usr/bin/env python3
""" This line indicates to the system to use Python3 """
import rclpy # rclpy is the Python client library for ROS 2
from rclpy.node import Node

class MyNode(Node): # Class MyNode that inherits from Node

    def __init__(self):
        super().__init__("obstacle_avoidance") # Named the node "obstacle_avoidance"
        self.get_logger().info("Hello limo car")

def main(args=None):
    rclpy.init(args=args) # Initialization of ROS 2 communication

    # Start node creation
    node = MyNode() # creation of the node
    rclpy.spin(node) # This keeps the node alive indefinitely, until ctrl+c is pressed
    rclpy.shutdown() # Shutdown ROS 2 communication when stopped


if __name__ == "__main__":
    main()
```

We can run the node with : **./safety_filter.py.** But to be able to use Ros2 ressources we must settle the s**etup.py** of the **package robot_security**. We configure it, we add **"obstacle_avoidance = robot_security.safety_filter:main"** at the line 23 as following.

```python
from setuptools import setup

package_name = 'robot_security'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='agilex',
    maintainer_email='agilex@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            "obstacle_avoidance = robot_security.safety_filter:main"
        ],
    },
)
```

To run the node, we must follow the following steps :

1- cd ~/ros2_kelly/ros2_ws_foxy
2- colcon build –symlink-link –packages-select robot_security
3- source install/setup.bash
4- ros2 run robot_security obstacle_avoidance

we can compile with colcon build only but at each time we change a line in safety_filter.py we must recompile again ; and with colcon build –symlink-install we don’t need to compile again the update is automatically done.

Algorithm for filtering :

We want to get all the distance arround the car as the Lidar is supposed to give 360 data for the 360 differents angles. To get these data and treat them, we write a Python code and the result was not what we expected, below is the result :

```
[INFO] [1780627647.929625800] [obstacle_avoidance]: for i = 225 Obstacle detected at 0.25 meters
[INFO] [1780627647.930207142] [obstacle_avoidance]: for i = 226 Obstacle detected at 0.25 meters
[INFO] [1780627647.930826085] [obstacle_avoidance]: for i = 227 Obstacle detected at 0.25 meters
[INFO] [1780627647.931460389] [obstacle_avoidance]: for i = 228 Obstacle detected at 0.25 meters
[INFO] [1780627647.932070340] [obstacle_avoidance]: for i = 229 Obstacle detected at 0.25 meters
[INFO] [1780627647.932663778] [obstacle_avoidance]: for i = 230 Obstacle detected at 0.24 meters
[INFO] [1780627647.933273473] [obstacle_avoidance]: for i = 231 Obstacle detected at 0.24 meters
[INFO] [1780627647.933873824] [obstacle_avoidance]: for i = 232 Obstacle detected at 0.24 meters
[INFO] [1780627647.934459709] [obstacle_avoidance]: for i = 233 Obstacle detected at 0.24 meters
[INFO] [1780627647.935087741] [obstacle_avoidance]: for i = 234 Obstacle detected at 0.24 meters
[INFO] [1780627647.935715837] [obstacle_avoidance]: for i = 235 Obstacle detected at 0.24 meters
[INFO] [1780627647.936314043] [obstacle_avoidance]: for i = 236 Obstacle detected at 0.24 meters
[INFO] [1780627647.936949116] [obstacle_avoidance]: for i = 237 Obstacle detected at 0.24 meters
[INFO] [1780627647.937544954] [obstacle_avoidance]: for i = 238 Obstacle detected at 0.00 meters
[INFO] [1780627647.938136952] [obstacle_avoidance]: for i = 239 Obstacle detected at 0.24 meters
[INFO] [1780627647.938725174] [obstacle_avoidance]: for i = 240 Obstacle detected at 0.25 meters
[INFO] [1780627647.939317812] [obstacle_avoidance]: for i = 241 Obstacle detected at 0.24 meters
[INFO] [1780627647.939930387] [obstacle_avoidance]: for i = 242 Obstacle detected at 0.24 meters
[INFO] [1780627647.940558867] [obstacle_avoidance]: for i = 243 Obstacle detected at 0.24 meters
[INFO] [1780627647.941187218] [obstacle_avoidance]: for i = 244 Obstacle detected at 0.00 meters
[INFO] [1780627647.941786833] [obstacle_avoidance]: for i = 245 Obstacle detected at 0.25 meters
[INFO] [1780627647.942374575] [obstacle_avoidance]: for i = 246 Obstacle detected at 0.24 meters
Traceback (most recent call last):
  File "/home/agilex/ros2_kelly/ros2_ws_foxy/install/robot_security/lib/robot_security/obstacle_avoidance", line 33, in <module>
    sys.exit(load_entry_point('robot-security', 'console_scripts', 'obstacle_avoidance')())
  File "/home/agilex/ros2_kelly/ros2_ws_foxy/build/robot_security/robot_security/safety_filter.py", line 52, in main
    rclpy.spin(node) # This keeps the node alive indefinitely, until ctrl+c is pressed
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/__init__.py", line 191, in spin
    executor.spin_once()
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/executors.py", line 719, in spin_once
    raise handler.exception()
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/task.py", line 239, in __call__
    self._handler.send(None)
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/executors.py", line 429, in handler
    await call_coroutine(entity, arg)
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/executors.py", line 354, in _execute_subscription
    await await_or_execute(sub.callback, msg)
  File "/opt/ros/foxy/lib/python3.8/site-packages/rclpy/executors.py", line 118, in await_or_execute
    return callback(*args)
  File "/home/agilex/ros2_kelly/ros2_ws_foxy/build/robot_security/robot_security/safety_filter.py", line 29, in scan_callback
    self.distance_forward = msg.ranges[i]
IndexError: array index out of range
```

Above i = 246, Lidar doesn't send data anymore, and we thought that Lidar can't send 360 valeur but only 247. In the code below, an error is displayed when I exceed 246 (line 28).

```
25
26      def scan_callback(self,msg: LaserScan):
27          #we listenning to capture the distance forward the car
28          for i in range(250):
29              self.distance_forward = msg.ranges[i]
30              self.get_logger().info(f"for i = {i} Obstacle detected at {self.distance_forward :.2f} meters")
```

we can access only 247 values coming from the Lidar. In deed, although the Lidar mesure 360 values some of them are not useful because they are the distance between the Lidar (representing the car) and some parts of the car while we want the distance between the Lidar and the obstacles.

We did a mapping in nav2.yaml by adding in controller_server the following part
remappings :
- /cmd_vel : /cmd_vel_nav

```
controller_server:
  ros__parameters:
    use_sim_time: False
    remappings:
      - /cmd_vel: /cmd_vel_nav
    controller_frequency: 10.0
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
    min_theta_velocity_threshold: 0.001
    controller_plugins: ["FollowPath"]

    # DWB parameters (Plus de tiret au-dessus !)
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      debug_trajectory_details: True
```

But the controller_server continued to pubish on /cmd_vel, to deal with it we have created our own navigation file called **custom_bringup_launc.py** in which we did the remapping, and as result it worked.
Finally, controller_server publish its command on /cmd_vel_nav we created and /cmd_vel_nav publish these command on /cmd_vel without filtering them in order to see if the communication works between the different nodes.

Minimal distance desired. :
forward : 0.13
left diagonale : 0.15
rigth diagonale : 0.15
left : 0.18
rigth : 0.18

USEFUL COMMANDS :

1- To list all the packages in the workspace :
colcon list --base-paths ~/ros2_ws/src
or colcon list , when we want the package in the current workspace.

2- To save a map (advanced command):
ros2 run nav2_map_server map_saver_cli -f ~/ros2_ws_foxy/maps/field --ros-args -p use_sim_time:=true

3- **To avoid taping the command « source /opt/ros/foxy/setup.bash » each time I want to use a container, I can add this command to the « .bashrc ». « .Bahsrc » is a set of command that automatically execute want a terminal opens.**
Echo ‘’source /opt/ros/foxy/setup.bash’’ >> ~/.bashrc
to charge the modifications we do : source ~/.bashrc

4- To see all the node involved in a communication :
ros2 run rqt_graph rqt_graph

5- To check if the lifecyle is activated :
ros2 lifecycle get /map_server configure
ros2 lifecycle get /map_server activate

6- Check the communication map→ odom :
ros2 run tf2_ros tf2_echo map odom

7- Domain ID :
echo $ROS_DOMAIN_ID

8- to put the container on the same domain id than the car :
docker exec -it -e ROS_DOMAIN_ID=2 limo_foxy_container bash

9- to copy the container workspace on the limo car :
docker cp limo_foxy_container:/root/ros2_ws_foxy ros2_kelly

10- to display the interface of the car on my computer :
ssh -X agilex@10.12.177.10

11- To scopy from docker toward limo car :
scp

12- To launch the whole programs on Limo car :
ros2 launch limo_base limo_base.launch.py

13- To control from the keyboard :
ros2 run teleop_twist_keyboard teleop_twist_keyboard

14- To activate the sensors :
ros2 launch limo_bringup limo_start.launch.py

15- To make a cartographie :
ros2 launch limo_bringup cartographer.launch.py

16- To compile the work :
cd /home/agilex/limo_ros2_ws
colcon build

17- start navigation :
ros2 launch limo_bringup limo_nav2.launch.py

Or if it is a Ackermann motion mode :
ros2 launch limo_bringup limo_nav2_ackmann.launch.py

18- important path
/home/agilex/limo_ros2_ws/install/... : wokspace

/home/agilex/limo_ros2_ws/src/...: Result of compilation

/opt/ros/foxy/… : general tools of ROS2

19- sourcing :
the global system :source /opt/ros/foxy/setup.bash
the workspace : source /home/agilex/limo_ros2_ws/install/setup.bash

20 – Saving map :
cd /home/agilex/limo_ros2_ws/src/limo_ros2/limo_bringup/maps
ros2 run nav2_map_server map_saver_cli -f map_name


LIMO PRO
UNIVERSITÉ & RECHERCHE
AGILE·X Robotics
Dimensions
322 x 220 x 251 mm
Compatibilité
ROS1 Noetic / ROS2 Foxy + Gazebo
Autonomie
2,5 h (in use)
4 h (en veille)
CPU
Nvidia Jetson Orin Nano 8G
Poids
4,8 kg
LiDAR
EAI T-mini Pro
Vitesse
3,6 km/h
Caméra de profondeur
Orbbec DaBai
LIMO APPLICATIONS
Mapping
Navigation
Obstacles avoidance
Paths planning
SLAM & V-SLAM
ROUES INTERCHANGEABLES
4 mecanum wheels
Ackermann
4 wheels differential steering
Tracked
COMPATIBLE ADD-ON
Piste de simulation
LIMO
02
VOIR LA FICHE PRODUIT
VOIR LA VIDEO

---

nav2.yaml : **/home/agilex/limo_ros2_ws/src/limo_ros2/limo_bringup/param/nav2.yaml**
github :https://github.com/agilexrobotics/limo_pro_doc/blob/master/Limo%20Pro%20Ros2%20Foxy%20user%20manual(EN).md